Macros that Work Together

Compile-Time Bindings, Partial Expansion, and Definition Contexts

Matthew Flatt[1], Ryan Culpepper[1], David Darais[1], and Robert Bruce Findler[2]
[1]*University of Utah* [2]*Northwestern University*

Tuesday, February 21st, 2012

Abstract

Racket is a large language that is built mostly within itself. Unlike the usual approach taken by non-Lisp languages, the self-hosting of Racket is not a matter of bootstrapping one implementation through a previous implementation, but instead a matter of building a tower of languages and libraries via macros. The upper layers of the tower include a class system, a component system, pedagogic variants of Scheme, a statically typed dialect of Scheme, and more. The demands of this language-construction effort require a macro system that is substantially more expressive than previous macro systems. In particular, while conventional Scheme macro systems handle stand-alone syntactic forms adequately, they provide weak support for macros that share information or macros that use existing syntactic forms in new contexts.

This paper describes and models features of the Racket macro system, including support for general compile-time bindings, sub-form expansion and analysis, and environment management. The presentation assumes a basic familiarity with Lisp-style macros, and it takes for granted the need for macros that respect lexical scope. The model, however, strips away the pattern and template system that is normally associated with Scheme macros, isolating a core that is simpler, can support pattern and template forms themselves as macros, and generalizes naturally to Racket's other extensions.

1 Macros as a Compiler-Extension API

The progression from text pre-processors (such as the C pre-processor) to Lisp macros to Scheme macros is an evolution toward a wider compiler API—one that, at the Scheme end, exposes the compiler's management of lexical context. This widening of the API makes certain language extensions possible that were technically impossible before, such as a local transformer that reliably expands to a reference of an enclosing binding.

The classic example of a scope-respecting macro is `or`, which (in simplified form) takes two expressions. It returns the value of the first expression if it is not `#f` (i.e., false) or the value of the second expression otherwise:

```
(or e1 e2) => (let ([tmp e1]) (if tmp tmp e2))
```

The `tmp` binding in the expansion of `or` ensures that the first expression is evaluated only once. A Scheme macro system ensures that the `or` macro works as expected in a setting like this expression:

```
(let ([tmp 5]) (or #f tmp))
```

An expansion oblivious to scope would allow the `or`-introduced `tmp` binding to shadow the outer binding of `tmp`; the program would then produce `#f` (false) instead of `5`. Instead, Scheme's hygienic macro expander (Kohlbecker et al. 1986) preserves the original apparent binding structure, and the `or`-introduced `tmp` does not shadow the outer `tmp`.

Although Scheme is best known for its pattern-matching macros (Clinger and Rees 1991; Kohlbecker and Wand 1987), the crucial addition in Scheme's macro API compared to Lisp is the *syntax object* data type (Dybvig et al. 1993; Sperber 2007), along with an operator for quoting literal program fragments. A syntax object represents a program fragment and carries with it information needed to respect lexical scope. The `#'` quoting operator is like the `'` operator, but `#'` produces a syntax object that encapsulates the program fragment's *lexical context*—the bindings in scope where the quoted fragment occurs. For example, the syntax object produced by

```
(let ([x 1]) #'x)
```

records that the program fragment occurred in the context of that particular binding of `x`. That lexical context information can be queried by functions such as `free-identifier=?`, which determines whether two identifiers are bound in the same place:

```
> (let ([x 1]) (free-identifier=? #'x #'x))
#t
> (free-identifier=? (let ([x 1]) #'x)
                     (let ([x 1]) #'x))
#f
```

Functions like `free-identifier=?` are typically used within *procedural macros*, which are bound with `define-syntax` and can be arbitrary functions that transform a source syntax object into a new syntax object.

Racket builds on procedural macros and syntax objects while further expanding the compiler functionality that is available through macros. The Racket macro API exposes the compiler's general capability to bind and access compile-time information within a lexical scope, as well as the compiler's ability to expand a sub-expression's macros. This wider macro API enables language extensions that were technically impossible (or, at best, awkward to simulate) in the narrower macro API of earlier Scheme systems. Such extensions can be generally characterized as macros that cooperate by sharing compile-time information, and we describe several examples in Section 2.

Section 3, which is the bulk of the paper, presents a model of Racket macros. The full model is about three pages long. For its presentation, we build up the model in a way that imitates the historical evolution of macro systems. We start with a core language and basic parsing rules, then add scope-oblivious macros, next add tracking of lexical scope within syntax objects, and finally add support for sub-form expansion and definition contexts.

2 Cooperating Macros

Macros in Racket cooperate with each other in many different ways, including the way that `define-struct` provides information for the `match` form, the way that the `class`

form leverages `define` and `lambda`, and the way that `lambda` propagates information about definitions within its body to later definitions. These uses illustrate key tools for cooperation: compile-time bindings, sub-form expansion (both complete and partial), and definition contexts.

2.1 Structure Definition and Matching

Whereas Scheme has just one notion of compile-time information, the macro, Racket supports the binding of identifiers to arbitrary compile-time information. One such example is structure information, which is how `define-struct` communicates information about the shape of a structure declaration to the `match` pattern-matching form.

The `define-struct` form expands to a set of run-time definitions using `define`, plus a single a compile-time binding using `define-syntax`. For example,

```
(define-struct egg (color size))
```

expands to the following definitions:

```
(define (make-egg c s) ....)
(define (egg? v) ....)
(define (egg-color e) ....)
(define (egg-size e) ....)
(define-syntax egg
  (make-struct-desc #'make-egg #'egg? ....))
```

The `make-egg` function is a constructor, the `egg?` function is a predicate, the `egg-color` function is a selector, and so on. The `egg` binding, meanwhile, associates a static description of the structure type—including references to its constructor, predicate, and selector functions—with the name `egg` for use in other macros. In general, the use of `define-syntax` does not always create a macro. If the value bound to the identifier introduced by `define-syntax` is a function, then the macro expander knows to call that function when it sees the identifier, but if the identifier is bound to something else, then using the identifier results in a syntax error.

Cooperating macros can, however, use the `syntax-local-value` function to extract the value bound to the identifier. In particular, the `match` pattern-matching form recognizes bindings to structure definitions using `syntax-local-value`, and it generates code that uses the predicate and selector functions. For example,

```
(define (blue-egg-size v)
  (match v
    [(egg 'blue s) s]))
```

expands to roughly

```
(define (blue-egg-size v)
  (if (and (egg? v) (eq? (egg-color v) 'blue))
      (egg-size s)
      (error "match: no matching case")))
```

The implementation of `match` uses `syntax-local-value` on the `egg` identifier to learn about its expected number of fields, its predicate, and its selector functions.

Using `define-syntax` for both macros and other compile-time bindings allows a single identifier to play multiple roles. For example, `make-struct-desc` in the expansion of Racket's `define-struct` macro produces a value that is *both* a structure description and a function.[1] Since the descriptor is also a function, it can act as a macro transformer when `egg` is used as an expression. The function behavior of a structure descriptor is to return the identifier of structure's constructor, which means that `egg` as an expression is replaced by the `make-egg` constructor; that is, `(egg 'blue 1)` expands to `(make-egg 'blue 1)`. Overloading the `egg` binding in this way allows `egg`-constructing expressions and `egg`-matching patterns to have the same shape.

2.2 Patterns and Templates

Macro transformers typically pattern match on uses of the macro to generate the macro's expansion. Although transformers could use the `match` form to match macro uses, Racket provides the `syntax-case` pattern-matching form, which is more specialized to the task of matching syntax fragments. The `syntax-case` form matches syntax objects, and the associated `syntax` form produces a syntax object using pattern variables that are bound by `syntax-case`. Arbitrary Racket code can occur between `syntax-case`'s binding of pattern variables and the `syntax` templates that use them.

For example, the following implementation of `defthunk` expands a use like `(defthunk f (random))` to `(define (f) (random))`. The macro transformer receives a use of `defthunk` as an `in-stx` argument, which is a syntax object. The `syntax-case` form attempts to match `in-stx` to the pattern `(defthunk g e)`, which matches when a use of `defthunk` has exactly two sub-forms; in that case, `g` is bound to the first sub-form and `e` is bound to the second:

```
(define-syntax defthunk
  (lambda (in-stx)
    (syntax-case in-stx ()
     [(defthunk g e)
      (if (identifier? (syntax g))
          (syntax (define (g) e))
          (error "need an identifier"))])))
```

The `(syntax g)` expression in the matching clause refers to the part of `in-stx` that matched `g` in the pattern, and the macro transformed checks that the `g` part of a use `in-`

[1] A structure description is itself a structure, and a structure can have a `prop:procedure` property that determines how the structure behaves when applied to arguments.

`stx` is an identifier. If so, the matching pieces `g` and `e` are used to assemble the macro expansion.

A challenge in implementing `syntax-case` and `syntax` is communicating the pattern variables bound by `syntax-case` to the uses in a `syntax` template. Since the right-hand side of a `syntax-case` clause can be an arbitrary expression, `syntax-case` cannot easily search for uses of `syntax` and replace pattern variables with match references. One way to handle this problem is to build `syntax-case` and `syntax` (or, at least, the notion of pattern variables) into the macro system. With generalized compile-time bindings like those in Racket, however, `syntax-case` can be implemented instead as a macro that binds each pattern variable to compile-time information describing how to access the corresponding matched value, and `syntax` checks each identifier in a template to determine whether it refers to such compile-time information.

For example, the `syntax-case` clause above is translated to the following:

```
(let ([tmp-g ... extract g from in-stx ...]
      [tmp-e ... extract e from in-stx ...])
  (if (and tmp-g tmp-e) ; if the pattern matched...
      (let-syntax ([g (make-pattern-var (syntax tmp-g) 0)]
                   [e (make-pattern-var (syntax tmp-e) 0)])
        (if (identifier? (syntax g))
            (syntax (define (g) e))
            (error "need an identifier")))
      (error "bad syntax")))
```

The `g` and `e` pattern variables are represented by compile-time records that contain references to the `tmp-g` and `tmp-e` variables that store the matched sub-forms. The records also store the ellipsis depth (Kohlbecker and Wand 1987) of the pattern variables, so that `syntax` can report mismatches at compile time. The `syntax` form checks each identifier in its template; if it is bound to a compile-time pattern variable record, it is translated to a reference to the corresponding run-time variable; otherwise, it is preserved as a literal syntax object. The inner `if` from therefore expands to

```
(if (identifier? tmp-g)
    (datum->syntax (list #'define (list tmp-g) tmp-e))
    (error "need an identifier"))
```

where the `datum->syntax` primitive converts list structure into a syntax object.

2.3 Classes, Definitions, and Functions

The syntax of a Racket `class` expression is

```
(class superclass-expr decl-or-expr*)
```

The *`superclass-expr`* can be the built-in `object%` class[2] or any other class, but the

[2] In Racket, class names traditionally end in `%`.

decl-or-expr sequence is our primary interest. The sequence declares all of the fields and methods of the class, in addition to expressions that are evaluated when the class is instantiated (analogous to a constructor body).

A typical use of the `class` form defines some private fields and public methods. To make the syntax of `class` easier for Racket programmers to remember, the syntax for such declarations within a class builds on the standard `define` form normally used to define variables and functions. For example,

```
(define chicken%
  (class object%
    (define eggs empty)
    (public nesting? lay-egg)
    (define (nesting?)
      (not (empty? eggs)))
    (define (lay-egg color size)
      (set! eggs (cons (make-egg color size)
                       eggs)))
    ....))
```

defines a class `chicken%` that has a private field `eggs` and public methods `nesting?` and `lay-egg`.

More than making the syntax easier to remember, reusing `define` for field and method declarations means that syntactic forms that expand to `define` also can be used. For example, a variant of `define` might support optional arguments by expanding to the plain `define` form:

```
(define/opt (lay-egg [color 'brown] [size 3])
  (set! eggs (cons (make-egg color size)
                   eggs)))
```

As another example, programmers using `class` often use a `define/public` form to declare a public method, instead of writing separate `define` and `public` forms. The `define/public` form expands to a sequence of `public` and `define` declarations.

Finally, although it is implicit in the function-shorthand uses of `define` above, the `class` form also reuses `lambda` for method declarations. For example, the `nesting?` method could have been written

```
(define nesting?
  (lambda () (not (empty? eggs))))
```

Similar to `define`, any macro that expands to `lambda` can be used with a `define` (or a macro that expands to `define`) to describe a method.

In order for the `class` macro to properly expand, it must be able to detect all bindings and functions in its body. Specifically, the macro must see all definitions to build a table of fields and methods, and it must see the functions that implement methods so that it can insert the implicit `this` argument (which a method receives when it is called) into the method's argument list. Thus, to allow the use of declaration forms like `de-`

`fine/public`, the `class` macro must force the expansion of each *`decl-or-expr`* to expose the underlying uses of `define`, `lambda`, and `public`.

Scheme macro systems do not typically provide a way to force expansion of a sub-form in the way that `class` requires. Sub-forms are normally expanded only once they appear directly within a core syntactic form, after all of the surrounding macros have been expanded away. That is, when a macro transformer returns an expression that contains macro uses, then the sub-expression macros are expanded iteratively. The `class` form, however, needs to force expansion of its sub-forms before producing its result.

The `class` form forces sub-expression expansion using the Racket `local-expand` function. The `local-expand` function takes a syntax object to expand, along with other arguments to be described later, and it returns the expanded form as a syntax object. The resulting syntax object can be inspected, transformed, and incorporated into a larger result by the macro transformer.

2.4 Internal Definitions

The reuse of `define` in `class` has a precedent in standard Scheme: `define` can be used inside `lambda` and other block forms to create local definitions. For example,

```
(define (cook eggs)
  (define total-size (sum-eggs eggs))
  (if (< total-size 10)
      (cook-in-small-pan eggs)
      (cook-in-big-pan eggs)))
```

creates a local binding `total-size` that is available only with the function body. Local definitions like this are called *internal definitions*.

In a fully expanded program, internal definitions can be replaced with a `letrec` local binding form.[3] The process of macro expansion must somehow discover and convert internal definitions to `letrec` forms. Complicating this process, an internal definition can bind a macro instead of a run-time variable, or an internal definition can shadow the binding of an identifier from the enclosing environment. Each of those cases can affect the expansion of later forms in a function body, even affecting whether the form is treated as an internal definition or as an expression, as in the following case:

```
(define (cook-omelette eggs)
  (define-syntax-rule (define-box id)
    (define id (box empty)))
  (define-box best-eggs)
  (define-box left-overs)
  (take-best-eggs! eggs best-eggs leftovers)
  (values (make-omelette (unbox best-eggs))
          rest-eggs))
```

[3] In the current Scheme standard (Sperber 2007), internal definitions are converted to a `letrec*` form. Racket's `letrec` form corresponds to the standard's `letrec*` form.

To handle the interaction of internal definitions and expressions, a syntactic form that allows internal definitions must partially expand each of its body sub-forms to determine which are definitions. Each macro definition must be installed immediately for use in expanding later body forms. If partial expansion reveals a run-time definition, expansion of the right-hand side of the definition must be delayed, because it might refer to bindings created later in the body (e.g., a forward reference to a function or macro that is defined later in the body).

These issues are typically resolved internal to a Scheme macro expander (Ghuloum and Dybvig 2007; van Tonder 2007), so that only built-in forms like `lambda` can accommodate internal definitions. Racket gives a macro transformer all of the tools it needs to implement internal-definitions contexts: partial sub-form expansion, an explicit representation of definition contexts, and an operation to extend a definition context with bindings as they are discovered. Consequently, a `lambda` form that supports internal definitions can be implemented in terms of a simpler `lambda` that allows only expressions in its body. Similarly, the `class` form can support local macros among its field and method definitions, or a `lambda` variant can support definitions mixed with expressions in its body (instead of requiring all definitions first, as in the standard Scheme `lambda` form).

To perform *partial expansion* of their sub-forms, the `lambda` and `class` macros provide `local-expand` with a *stop list*, a list of identifiers to use as stopping points in expansion. For `lambda`, the stop list includes only the core syntactic forms, ensuring that all definition-producing macros are expanded into core definitions. The Racket specification pins down the set of core syntactic forms, and the corresponding identifiers are assembled in a library-provided list, which is sufficient to make most macros cooperate properly. The `class` macro uses a stop list that also includes identifiers like `#'public` and `#'override`, since those forms must be caught and interpreted by the `class` macro; they are meaningless to the Racket macro expander.[4] When macro uses nest and the corresponding transformers use partial expansion, the inner transformer's partial expansion is not affected by the stop list of the outer transformer, so macros need not be aware of the stop lists of other macros.

To support internal definitions, the `lambda` and `class` macros generate a new *definition context* value using the `syntax-local-make-definition-context` function. The macros provide this context value to `local-expand` along with the stop list to partially expand the body forms in the scope of the definitions uncovered so far. When the `lambda` or `class` macros detect a new definition via partial expansion, they install new bindings into the definition context using `syntax-local-bind-syntaxes`. When the macros detect a `define` form, they call `syntax-local-bind-syntaxes` with just the defined identifiers, which are added to the definition context as bindings for run-time variables. When the macros detect a `define-syntax` form, they call `syntax-local-bind-syntaxes` with identifiers and the corresponding compile-time expression, which is evaluated and associated with the identifiers as compile-time bindings.

[4] By convention, identifiers such as `public` are bound as macros that raise a syntax when used incorrectly—that is, outside of a `class` body.

2.5 Packages

Definition contexts and compile-time binding further enable the implementation of a local-module form as a macro. Racket's `define-package` form resembles the `module` form from Chez Scheme (Waddell and Dybvig 1999) and the `structure` form of ML (Milner et al. 1990). A set of definitions within a package can see each other, but they are hidden from other expressions. Exported identifiers listed after the package name become visible when the package is explicitly opened:

```
(define-package carton (eggs)
  (define egg1 (make-egg 'blue 1))
  (define egg2 (make-egg 'white 2))
  (define eggs (list egg1 egg2)))
....
(open-package carton)
```

To allow definitions within a package to see each other, the `define-package` form creates a definition context for the package body. The definition context does not escape the package body, so no other expressions can directly access the package contents. Meanwhile, the package name is bound to a compile-time description of the contents, so that `open-package` can make the exported names available in a later scope, and the package name itself can be exported and imported like any other binding. When a package is opened with `open-package`, the package's names are made available by new `define-syntax` bindings that redirect to the package's hidden definitions.

Naturally, packages can be defined within packages, which is supported in the macro API by allowing definition contexts to nest. Going even further, `define-package` supports a `define*` form that binds an identifier for only *later* expressions within the package body, like ML's nested `val` bindings instead of Scheme's mutually recursive `define` bindings. Such variations on binding scopes are possible in Racket because the machinery of definition contexts is exposed in the macro API.

2.6 Tools

The DrRacket programming environment includes many tools that manipulate Racket programs and modules, including a debugger, a profiler, and a syntax checker. These tools all work by first expanding the program, so that they need to handle only the core forms of the language. The tools are not macros, but they gain many of same sorts of benefits as cooperating macros by using an `expand` function that produces a syntax object.

A typical Scheme macro expander (Ghuloum and Dybvig 2007; van Tonder 2007) takes a syntax object and produces a raw S-expression (i.e., pairs and lists), but the `expand` function produces a syntax object for the expanded program. Through syntax objects, the original names of local variables are intact within an expanded program, while lexical-context information in the syntax object relates binding occurrences to bound uses. Another advantage is that various language extensions for manipulating syntax objects in macro transformers—notably the `syntax-case` form that gives the macro system its name—are also available for use by tools that process expanded programs.

Syntax objects thus serve as an intermediate representation of programs for all Racket

tools, whether they simply inspect the program (as in the syntax checker, to show the program's binding structure via arrows overlaid on the source text) or transform the program (as in the profiler, to add instrumentation). To allow the latter, in particular, the output of the `expand` function must also be a suitable input to `expand`, and `expand` must be idempotent.

3 Modeling Macro Expansion in Racket

This section builds up a formal model of Racket macro expansion. We build on a traditional Lisp perspective, instead of assuming previous Scheme models as background. In part, this strategy is aimed at making the presentation as widely accessible as possible, but it also lets us adjust and simplify some core representation and expansion details for Scheme-style macros.

We begin with a core functional language without macros. We then create a surface language and add syntax objects representing terms in the surface language to the set of core-language values. We progressively extend the model with naive macros, macros with proper lexical scoping, and macros that communicate.

We use the following terminology to describe the relationships between the different components. The *reader* consumes a surface program in textual form and produces its representation as a syntax object. That representation is recursively *expanded* until all macros have been eliminated; the result is a "fully-expanded" syntax object. Finally, the fully-expanded syntax object is *parsed* into a core-language AST, which may be evaluated.

The sequence of models is implemented and typeset using PLT Redex (Felleisen et al. 2009). The sources are available from our web site:

`http://www.cs.utah.edu/plt/expmodel-5/`

3.1 Core Language

The core language of our model includes variables, function applications tagged with **App**, and values. Values include functions formed with **Fun**, lists formed with **List**, symbols formed with a curly quote, primitive operations, and possibly other kinds of data.

ast ::= *var* | (**App** *ast ast* ...) | *val*
var ::= (**Var** *name*)
val ::= (**Fun** *var ast*) | (**List** *val* ...) | *atom*
atom ::= *sym* | *prim* | ....
sym ::= '*name*
prim ::= **`cons`** | **`car`** | **`cdr`** | **`list`** | ....
name ::= a token such as `x`, `egg`, or `lambda`

We represent a *var* as a *name* wrapped with a **Var** constructor. We use a **Var** constructor to help distinguish names in general from names that are used to represent variables, since names are also used in the representation of symbols.

Primitive operations are treated as literals for simplicity. For example, the term **`cons`** is the actual primitive operation itself, not a variable whose value is the operation. Primitive operations are applied using the same **App** form as for applying functions, so **App** allows multiple argument expressions, even though a **Fun** accepts only a single argument.

Evaluation of the core language is standard (using substitution for functions):

$$\begin{array}{ll}
\mathsf{eval}[\![(\textbf{App}\ (\textbf{Fun}\ \mathit{var}\ \mathit{ast}_{\mathit{body}})\ \mathit{ast}_{\mathit{arg}})]\!] & = \mathsf{eval}[\![\mathit{ast}_{\mathit{body}}[\mathit{var} \leftarrow \mathsf{eval}[\![\mathit{ast}_{\mathit{arg}}]\!]]]\!] \\
\mathsf{eval}[\![(\textbf{App}\ \mathit{prim}\ \mathit{ast}_{\mathit{arg}}\ ...)]\!] & = \delta(\mathit{prim}\ \mathsf{eval}[\![\mathit{ast}_{\mathit{arg}}]\!]\ ...) \\
\mathsf{eval}[\![(\textbf{App}\ \mathit{ast}_{\mathit{op}}\ \mathit{ast}_{\mathit{arg}}\ ...)]\!] & = \mathsf{eval}[\![(\textbf{App}\ \mathsf{eval}[\![\mathit{ast}_{\mathit{op}}]\!]\ \mathit{ast}_{\mathit{arg}}\ ...)]\!] \\
\mathsf{eval}[\![\mathit{val}]\!] & = \mathit{val}
\end{array}$$

The second case of eval defers the implementation of primitives to a δ relation, which covers at least the primitive operations on lists:

$$\begin{array}{ll}
\delta(\texttt{cons}\ \mathit{val}_1\ (\textbf{List}\ \mathit{val}_2\ ...)) & = (\textbf{List}\ \mathit{val}_1\ \mathit{val}_2\ ...) \\
\delta(\texttt{car}\ (\textbf{List}\ \mathit{val}_1\ \mathit{val}_2\ ...)) & = \mathit{val}_1 \\
\delta(\texttt{cdr}\ (\textbf{List}\ \mathit{val}_1\ \mathit{val}_2\ ...)) & = (\textbf{List}\ \mathit{val}_2\ ...) \\
\delta(\texttt{list}\ \mathit{val}\ ...) & = (\textbf{List}\ \mathit{val}\ ...) \\
... &
\end{array}$$

The language can contain other primitive operations, such as **+**, which are also given meaning through δ.

3.2 Syntax Objects

The core language serves two roles: it is the target language into which a surface program is parsed, and it is also the language for implementing macro transformers. Consequently, the values of the core language must include representations of the surface-language fragments that macro transformers manipulate; that is, syntax objects.

To model syntax objects, we extend the core language's values with syntax objects and primitive operations on syntax objects:

$$\begin{array}{rl}
\mathit{val} ::= & (\textbf{List}\ \mathit{val}\ ...) \mid \mathit{atom} \mid \mathit{stx} \mid .... \\
\mathit{prim} ::= & \texttt{stx-e} \mid \texttt{mk-stx} \mid ....
\end{array}$$

The new primitive **stx-e** is short for Racket's `syntax-e`, and **mk-stx** is short for `make-syntax`.

Syntax objects, tagged with **Stx**, combine a value with lexical-context information *ctx*. The value must be either an atom or a list of syntax objects. We introduce lexical-context information later, and for now just use • for *ctx*.

$$\begin{array}{rl}
\mathit{stx} ::= & (\textbf{Stx}\ \mathit{atom}\ \mathit{ctx}) \mid (\textbf{Stx}\ (\textbf{List}\ \mathit{stx}\ ...)\ \mathit{ctx}) \\
\mathit{id} ::= & (\textbf{Stx}\ \mathit{sym}\ \mathit{ctx}) \\
\mathit{ctx} ::= & \bullet
\end{array}$$

The set of identifiers *id* is a subset of *stx*, including only those syntax objects that wrap a symbol.

3.2.1 Names, Variables, Symbols, and Identifiers

The terms *name*, *variable*, *symbol*, and *identifier* are easily confused, but we use each term in a specific way. To recap,

- A *name*, such as `x`, is a member of some abstract set of tokens in the meta-language (i.e., a "meta-symbol" in the implementing language, as opposed to a symbol in the implemented language). Names are used in the representation of variables and symbols.

- A *variable*, such as (**VAR** `x`), is the formal argument of a function, or it is a reference to a function argument that is replaced by a value during evaluation. Variables appear only in ASTs.
- A *symbol*, such as '`x`, is a value during evaluation. A symbol can appear as a literal expression, but since a symbol is constructed using a curly quote, it is never mistaken for a variable and replaced with a value.
- An *identifier*, such as (**STX** '`x` •), is a symbol combined with a lexical context. Like a symbol, an identifier is a value during evaluation—especially during the evaluation of macro transformers.

A Lisp programmer may be tempted to think of variables as implemented with symbols. Indeed, when an interpreter is implemented in Lisp, a variable or symbol in the interpreted language is typically represented using a symbol in the interpreter. Our eval, in contrast, is a mathematical function; variables and symbols are therefore implemented by names, which are entities in the mathematical world where the eval function resides. We highlight the distinction between language and meta-language to clarify which concepts are inherently connected within the language (e.g., symbols and identifiers) and which are related only by a representation choice in the meta-language (e.g., symbols and variables, both as names).

3.2.2 Readers and Syntax Objects

A *reader* consumes a textual representation of a surface program and produces a corresponding syntax object. For example, the reader would convert the source program

```
(lambda x x)
```

into its representation as a syntax object,

(**STX** (**LIST** (**STX** '`lambda` •) (**STX** '`x` •) (**STX** '`x` •)) •)

We do not model the reader process that takes a sequence of characters for a source program and converts it into a value that represents the source; we work only with the syntax-object representation.

The following extension of δ models the new primitive **`stx-e`** and **`mk-stx`** operations on syntax objects:

$$\begin{aligned}
&\delta(\textbf{stx-e}\ (\textbf{STX}\ \mathit{val}\ \mathit{ctx})) &&= \mathit{val}\\
&\delta(\textbf{mk-stx}\ \mathit{atom}\ (\textbf{STX}\ \mathit{val}\ \mathit{ctx})) &&= (\textbf{STX}\ \mathit{atom}\ \mathit{ctx})\\
&\delta(\textbf{mk-stx}\ (\textbf{LIST}\ \mathit{stx}\ ...)\ (\textbf{STX}\ \mathit{val}\ \mathit{ctx})) &&= (\textbf{STX}\ (\textbf{LIST}\ \mathit{stx}\ ...)\ \mathit{ctx})\\
&\ldots
\end{aligned}$$

That is, **`stx-e`** unwraps a syntax object by throwing away its immediate context, while **`mk-stx`** constructs a new syntax object by borrowing the context from an existing syntax object (which might have been a literal **STX** value in the original program or might itself have been constructed with **`mk-stx`**).

For example,

$$\begin{aligned}
&\text{eval}[\![(\textbf{APP}\ \textbf{stx-e}\ (\textbf{APP}\ \textbf{mk-stx}\ \text{'x}\ (\textbf{STX}\ \text{'y}\ \bullet)))]\!]\\
&= \delta(\textbf{stx-e}\ \text{eval}[\![(\textbf{APP}\ \textbf{mk-stx}\ \text{'x}\ (\textbf{STX}\ \text{'y}\ \bullet))]\!])\\
&= \delta(\textbf{stx-e}\ \delta(\textbf{mk-stx}\ \text{'x}\ (\textbf{STX}\ \text{'y}\ \bullet)))\\
&= \delta(\textbf{stx-e}\ (\textbf{STX}\ \text{'x}\ \bullet))\\
&= \text{'x}
\end{aligned}$$

3.2.3 Model vs. Implementations

The core model's **FUN** AST form is close to `lambda` in Scheme, and the *sym* representation is similar to a quoted symbol in Scheme. The model's **`list`** primitive operation is analogous to a `list` function, while a **LIST** constant is more like a quoted list in Scheme. For example, the model AST

(**LIST** '`lambda` (**LIST** '`x`) '`y`)

is analogous to the Scheme expression

```
'(lambda (x) y)
```

Along the same lines, an **STX** literal in the model AST is analogous to a syntax-quoted form in Scheme. For example,

(**STX** (**LIST** (**STX** '`lambda` •) (**STX** (**LIST** (**STX** '`x` •)) •) (**STX** '`y` •)) •)

is analogous to

```
#'(lambda (x) y)
```

where `#'` is a shorthand for a `syntax` form in the same way that `'` is a shorthand for a `quote` form. Note that in Racket, the printed form of a syntax object reports its source location (if any) and the encapsulated expression text:

```
> #'(lambda (x) y)
#<syntax:1:0 (lambda (x) y)>
```

The **`stx-e`** model primitive corresponds to the `syntax-e` function in Racket, and **`mk-stx`** in the model is similar to `datum->syntax` with its arguments reversed:

```
> (syntax-e (datum->syntax #'y 'x))
'x
```

Applying `syntax-e` to a complex syntax object exposes pieces that might be manipulated with `car` and `cdr`. Often, such pieces are reassembled with `datum->syntax`:

```
> (define stx #'(fun x y))

> (syntax-e stx)
'(#<syntax:1:0 fun> . #<syntax:1:0 (x y)>)
> (datum->syntax stx
                 (cons #'lambda
                       (cdr (syntax-e stx))))
#<syntax (lambda x y)>
```

The model is simpler and more primitive than Racket and Scheme in several ways. The `datum->syntax` function recurs into a list whose elements are not syntax objects, which is why the model's non-recurring **`mk-stx`** has a different name. In Racket, the core form without pattern variables is `quote-syntax`, and `syntax` expands to `quote-syntax` for literal program fragments. Standard `syntax-case` systems do not include

Racket's `syntax-e` operation, although it is essentially the expose function from Dybvig et al. (1993); instead, the built-in pattern-matching notation is used to deconstruct syntax objects. The `syntax->datum` operation, meanwhile, recursively applies `syntax-e`, discarding lexical-context information both on the immediate syntax object and on nested syntax objects.

Not all implementations of `syntax-case` associate a lexical context to a list or number. In Racket, consistently associating a lexical context to every program fragment gives the programmer control over the expansion of constants and application forms. Such control is beyond the scope of this paper, but our model is intended to accommodate those extensions, which are used heavily in the implementation of Racket (e.g., to support functions with keyword arguments).

3.3 Parsing

Parsing is the task of converting a syntax object to an AST that can be evaluated. We define a parser for a Scheme-like language as follows:

- A `lambda` form is parsed into a **FUN** *ast* node. Unlike in Scheme, `lambda` allows only a single argument and omits a set of parentheses around the argument.
- All literal values, even primitive operations, must be `quote`d in a source program; the quoted literals are parsed as *atom*s.
- A `syntax` form is parsed into an *stx* value (without support for pattern variables).
- A sequence of expressions grouped with parentheses is parsed as an **APP** node when the first element of the group is not the name of a primitive syntactic form (such a `lambda` or `quote`) or a macro.
- An identifier as an expression is parsed as a *var*.

For example, a function that accepts a single number argument to increment would be written in the surface language as

```
(lambda x ('+ x '1))
```

which the reader converts to the *stx*

```
(STX (LIST (STX 'lambda •)
           (STX 'x •)
           (STX (LIST (STX (LIST (STX 'quote •)
                                 (STX + •)) •)
                      (STX 'x •)
                      (LIST (STX 'quote •)
                            (STX 1 •)))
                •))
     •)
```

and the job of the parser is to convert this *stx* to the *ast*

```
(FUN (VAR x) (APP + (VAR x) 1))
```

3.3.1 Symbol-Driven Parser

Ignoring macros, and also assuming that keywords like `lambda` are never shadowed, we could implement a parser from *stx*es to *ast*s with the following parse meta-function:

$$\begin{array}{l}
\mathsf{parse}[\![(\textbf{STX}\ (\textbf{LIST}\ (\textbf{STX}\ \texttt{'lambda}\ \bullet)\ (\textbf{STX}\ \texttt{'}\mathit{name}\ \bullet)\ \mathit{stx})\ \bullet)]\!] = (\textbf{FUN}\ (\textbf{VAR}\ \mathit{name})\ \mathsf{parse}[\![\mathit{stx}]\!]) \\
\mathsf{parse}[\![(\textbf{STX}\ (\textbf{LIST}\ (\textbf{STX}\ \texttt{'quote}\ \bullet)\ \mathit{stx})\ \bullet)]\!] = \mathsf{strip}[\![\mathit{stx}]\!] \\
\mathsf{parse}[\![(\textbf{STX}\ (\textbf{LIST}\ (\textbf{STX}\ \texttt{'syntax}\ \bullet)\ \mathit{stx})\ \bullet)]\!] = \mathit{stx} \\
\mathsf{parse}[\![(\textbf{STX}\ (\textbf{LIST}\ \mathit{stx}_{\mathit{rator}}\ \mathit{stx}_{\mathit{rand}}\ ...)\ \bullet)]\!] = (\textbf{APP}\ \mathsf{parse}[\![\mathit{stx}_{\mathit{rator}}]\!]\ \mathsf{parse}[\![\mathit{stx}_{\mathit{rand}}]\!]\ ...) \\
\mathsf{parse}[\![(\textbf{STX}\ \mathit{name}\ \bullet)]\!] = (\textbf{VAR}\ \mathit{name})
\end{array}$$

The clauses to define meta-functions in this paper are ordered, so that the next-to-last clause of parse produces an **APP** form when the initial identifier in a sequence is not `lambda`, `quote`, or `syntax`.

The parse function uses a strip meta-function to implement `quote` by stripping away lexical context:

$$\begin{array}{l}
\mathsf{strip}[\![(\textbf{STX}\ \mathit{atom}\ \mathit{ctx})]\!] = \mathit{atom} \\
\mathsf{strip}[\![(\textbf{STX}\ (\textbf{LIST}\ \mathit{stx}\ ...)\ \mathit{ctx})]\!] = (\textbf{LIST}\ \mathsf{strip}[\![\mathit{stx}]\!]\ ...)
\end{array}$$

The difference between a `quote` form and a `syntax` form is that the latter does not strip lexical context from the input representation.

3.3.2 Identifier-Driven Parser

When we add lexical-context information to *stx* (instead of just using •), parse will need to take that information into account, instead of simply looking for identifiers named `lambda`, `quote`, and `syntax`. To prepare for that change, we refine parse as follows, deferring identifier resolution to a resolve meta-function. For now, resolve simply extracts the *name* in an identifier, but we will refine it later to use the lexical-context information of an identifier.[5]

$$\mathsf{resolve}[\![(\textbf{STX}\ \texttt{'}\mathit{name}\ \bullet)]\!] = \mathit{name}$$

$$\begin{array}{l}
\mathsf{parse}[\![(\textbf{STX}\ (\textbf{LIST}\ \mathit{id}_{\mathit{lambda}}\ \mathit{id}_{\mathit{arg}}\ \mathit{stx}_{\mathit{body}})\ \mathit{ctx})]\!] = (\textbf{FUN}\ (\textbf{VAR}\ \mathsf{resolve}[\![\mathit{id}_{\mathit{arg}}]\!])\ \mathsf{parse}[\![\mathit{stx}_{\mathit{body}}]\!]) \\
\quad \text{where } \texttt{lambda} = \mathsf{resolve}[\![\mathit{id}_{\mathit{lambda}}]\!] \\
\mathsf{parse}[\![(\textbf{STX}\ (\textbf{LIST}\ \mathit{id}_{\mathit{quote}}\ \mathit{stx})\ \mathit{ctx})]\!] = \mathsf{strip}[\![\mathit{stx}]\!] \\
\quad \text{where } \texttt{quote} = \mathsf{resolve}[\![\mathit{id}_{\mathit{quote}}]\!] \\
\mathsf{parse}[\![(\textbf{STX}\ (\textbf{LIST}\ \mathit{id}_{\mathit{syntax}}\ \mathit{stx})\ \mathit{ctx})]\!] = \mathit{stx} \\
\quad \text{where } \texttt{syntax} = \mathsf{resolve}[\![\mathit{id}_{\mathit{syntax}}]\!] \\
\mathsf{parse}[\![(\textbf{STX}\ (\textbf{LIST}\ \mathit{stx}_{\mathit{rator}}\ \mathit{stx}_{\mathit{rand}}\ ...)\ \mathit{ctx})]\!] = (\textbf{APP}\ \mathsf{parse}[\![\mathit{stx}_{\mathit{rator}}]\!]\ \mathsf{parse}[\![\mathit{stx}_{\mathit{rand}}]\!]\ ...) \\
\mathsf{parse}[\![\mathit{id}]\!] = (\textbf{VAR}\ \mathsf{resolve}[\![\mathit{id}]\!])
\end{array}$$

[5] Notation: We use "where" in the parse metafunction to write side conditions that more conventionally would be written with "if," whereas "where" more conventionally binds metavariables. In later metafunctions, we use "where" as in PLT Redex to generalize and unify conventional "where" and "if" clauses. That is, "where" is a pattern-matching form that binds italicized metavariables, and it also acts as a side condition by requiring a match.

The parse meta-function in our model serves the same role as the parse meta-function in the model of Dybvig et al. (1993). Unlike the Dybvig et al. (1993) model, where parse is mutually recursive with an expand meta-function, our parse function works only on fully-expanded terms, and we define a separate expansion process that both consumes and produces a syntax object. This difference paves the way for sub-form expansion (which must expand without parsing), and it also reflects the use of syntax objects as a general-purpose intermediate format in Racket (as discussed in Section 2.6).

3.4 Expansion

The next step to modeling Scheme macro expansion is to create an *expander* that takes a syntax object for a source program and returns a syntax object for the expanded program. The expander sits between the reader and the parser, so that it starts with a syntax object that may have macro definitions and uses, and it produces a syntax object that fits the limited shape of syntax objects that are recognized by the parser. In addition to recognizing macro definitions and uses, the expander will have to recognize all of the forms that the parser recognizes; it nevertheless defers the production of an AST to the parser, so that the result of the expander can be used for further expansion in some contexts.

Even without introducing macros, the expander has a role in preparing a source program: The parse meta-function assumes that a `lambda` identifier always indicates a function form, but we want our source language to be like Scheme, where any identifier can be used as a local variable name—even `lambda`. The expander, therefore, must rename formal arguments of a function to ensure that they do not shadow the identifiers that parse uses as keywords.

The expander is implemented as an expand meta-function. To handle shadowing, and eventually to handle macro bindings, a compile-time environment ξ is provided to each use of expand. This environment maps *name*s to *transformer*s, and expand normally starts with an environment that maps `lambda` to the FUN transformer, `quote` to the QUOTE transformer, and `syntax` also to the QUOTE transformer. (The parse meta-function treats `quote` and `syntax` differently, but they turn out to be the same at the level of expand.) A *transformer* also can be an identifier tagged with VAR, which represents a variable bound by an enclosing function.

ξ ::= a mapping from *name* to *transformer*
transformer ::= FUN | QUOTE | (VAR *id*)

Each case for expand is similar to a corresponding case in parse, except that `quote` and `syntax` are collapsed into a single case:

expand⟦(**Stx** (**List** id_{lam} id_{arg} stx_{body}) ctx), ξ⟧ = (**Stx** (**List** id_{lam} id_{new} $stx_{expbody}$) ctx)
 where Fun = ξ(resolve⟦id_{lam}⟧), $name_{new}$ = fresh, id_{new} = (**Stx** '$name_{new}$ •),
 ξ_{new} = ξ+{resolve⟦id_{arg}⟧ →(Var id_{new})}, $stx_{expbody}$ = expand⟦stx_{body}, ξ_{new}⟧
expand⟦(**Stx** (**List** id_{quote} stx) ctx), ξ⟧ = (**Stx** (**List** id_{quote} stx) ctx)
 where Quote = ξ(resolve⟦id_{quote}⟧)
expand⟦(**Stx** (**List** stx_{rator} stx_{rand} ...) ctx), ξ⟧ = (**Stx** (**List** $stx_{exprator}$ $stx_{exprand}$ ...) ctx)
 where $stx_{exprator}$ = expand⟦stx_{rator}, ξ⟧, $stx_{exprand}$ ... = expand⟦stx_{rand}, ξ⟧ ...
expand⟦id, ξ⟧ = id_{new}
 where (Var id_{new}) = ξ(resolve⟦id⟧)

A significant difference from parse is that the `lambda` case of expand generates a new name for the formal argument in a `lambda` form, which ensures that the expanded program does not use any parse-recognized names as variables. The `lambda` case maps the original name to the new one in the environment for the `lambda` form's body. Correspondingly, the case in expand for expanding a variable reference installs the new name in place of the original, which it finds by consulting the environment.

As an example, the source

```
(lambda lambda lambda)
```

expands to the identity function essentially as follows:

```
  expand⟦(lambda lambda lambda), ξ0⟧
= (lambda lambda2 expand⟦lambda, ξ0+{lambda→lambda2}⟧)
= (lambda lambda2 lambda2)
```

To make the expansion trace above more readable, identifiers are reduced to their resolve results, lexical-context information is dropped, ξ_0 stands for the initial environment, and other obvious simplifications are applied.

3.5 Binding and Using Macros

To support macros, we extend the source language with a `let-syntax` form that is a simplified version of Scheme's macro-binding forms. Our `let-syntax` will bind a single identifier to a macro transformer function for use in the `let-syntax` body expression. For example, the following source program defines and uses a `thunk` macro to delay evaluation of an expression until it is applied to a (dummy) argument:

```
(let-syntax
  thunk (lambda e
          ('mk-stx
           ('list (syntax lambda) (syntax a)
                  ('car ('cdr ('stx-e e))))
           e))
  ((thunk ('+ '1 '2)) '0))
```

The `e` argument to the macro transformer is the representation of the use of the macro `(thunk ('+ '1 '2))`. The transformer extracts the `('+ '1 '2)` sub-expression from this representation using **`stx-e`**, **`cdr`**, and **`car`** on `e`. The transformer then places

the sub-expression into a `lambda` expression using **`list`** and **`mk-stx`**, producing a representation of `(lambda a ('+ '1 '2))`.

Support for macros in the expander requires new cases for evaluating core-form expressions during the process of expansion. No changes are needed to *ast* or parse to support macros, however, since the expander eliminates all uses of macros. The new expander cases include all of the old cases, plus cases for macro bindings and macro applications. The macro-binding case implements the new LET-SYNTAX transformer:

$$\begin{array}{l}\mathsf{expand}[\![(\textbf{STX}\ (\textbf{LIST}\ id_{ls}\ id_{mac}\ stx_{rhs}\ stx_{body})\ ctx), \xi]\!] = \mathsf{expand}[\![stx_{body}, \xi_1]\!] \\ \quad \text{where LET-SYNTAX} = \xi(\mathsf{resolve}[\![id_{ls}]\!]), \\ \qquad \xi_1 = \xi + \{\mathsf{resolve}[\![id_{mac}]\!] \rightarrow \mathsf{eval}[\![\mathsf{parse}[\![stx_{rhs}]\!]]\!]\}\end{array}$$

In this case, to evaluate the right-hand side of a `let-syntax` form, the right-hand side is first parsed. Using parse directly reflects the fact that this model does not cover macro transformers that are implemented in terms of macros (except that a macro expansion can include uses of macros).[6] The parsed right-hand side is then evaluated, and the result is bound in the compile-time environment while the `let-syntax` body is expanded.

The case for a macro application is triggered when the compile-time environment maps a name to a function value. Invocation of the macro applies the value from the environment to the macro-use source form. After the macro produces a value (which must be a syntax object), the expander is again applied to the result.

$$\begin{array}{l}\mathsf{expand}[\![stx_{macapp}, \xi]\!] = \mathsf{expand}[\![\mathsf{eval}[\![(\textbf{APP}\ val\ stx_{macapp})]\!], \xi]\!] \\ \quad \text{where } (\textbf{STX}\ (\textbf{LIST}\ id_{mac}\ stx_{arg}\ ...)\ ctx) = stx_{macapp}, \\ \qquad val = \xi(\mathsf{resolve}[\![id_{mac}]\!])\end{array}$$

Because we have not yet added lexical-context information to syntax objects, the macro system at this point resembles a traditional Lisp `defmacro` system. For example, using the `thunk` macro as defined above, the expression

```
(((lambda a (thunk ('+ a '1))) '5) '0)
```

produces `1` instead of `6`, because the `a` binding introduced by the `thunk` macro captures `a` in the expression supplied to `thunk`. That is, the `thunk` macro does not respect the lexical scope of the original program. The expander produces this result for the `lambda` form roughly as follows, in an environment ξ_0 that maps `thunk` to the transformer:

$$\begin{array}{l}\quad \mathsf{expand}[\![(\texttt{lambda a (thunk ('+ a '1)))}, \xi_0]\!] \\ = (\texttt{lambda a2}\ \mathsf{expand}[\![(\texttt{thunk ('+ a '1)}), \xi_1 = \xi_0 + \{\texttt{a} \rightarrow \texttt{a2}\}]\!]) \\ = \textit{... calling the } \texttt{thunk} \textit{ transformer...} \\ = (\texttt{lambda a2}\ \mathsf{expand}[\![(\texttt{lambda a ('+ a '1)}), \xi_1]\!]) \\ = (\texttt{lambda a2 (lambda a3}\ \mathsf{expand}[\![(\texttt{'+ a '1}), \xi_2 = \xi_1 + \{\texttt{a} \rightarrow \texttt{a3}\}]\!])) \\ = \textit{... expanding the body, no more extensions to } \xi_2 \textit{...} \\ = (\texttt{lambda a2 (lambda a3 ('+ a3 '1)))}\end{array}$$

[6] Although expand could be applied to transfomer expressions using the current compile-time environment as in Dybvig et al. (1993), doing so mixes binding phases in a way that is not true to Racket or allowed by the current Scheme standard (Sperber 2007). The model is instead easily generalized to support expansion of transformer expressions through modules and phases (Flatt 2002).

3.6 Tracking Lexical Context

To change the macro system so that macro transformers respect lexical scope, we introduce lexical-context information into syntax objects.

3.6.1 Scope Examples

The first challenge in tracking binding through macro expansion is illustrated by the following example:

```
((lambda x
  (let-syntax m (lambda stx (syntax x))
    (lambda x
      ('+ (m) x))))
 1)
```

The expansion of `(m)` carries a reference to the outer `x` into the scope of the inner `x`. Proper lexical scoping demands that the two `x`s are kept distinct.

At first glance, the solution is simply to capture the compile-time environment ξ in either the `m` binding, the `(lambda stx (syntax x))` closure, or the `(syntax x)` syntax object. That way, when the `x` that is introduced by the expansion of `m` is further expanded, the captured environment is used instead of the current compile-time environment. The captured environment then correctly maps `x` to the binding from the outer `lambda`.

Although the intuition is appealing, a simple environment-capturing approach does not work in general, because identifiers introduced by a macro expansion can appear in binding positions as well as use positions. For example, in

```
(lambda x
  (let-syntax n (lambda stx
                  ; expand (n e) to (lambda x ('+ e x))
                  ('mk-stx
                   stx
                   ('list (syntax lambda) (syntax x)
                          ('mk-stx
                           stx
                           ('list (syntax '+)
                                  ('car ('cdr ('stx-e x)))
                                  (syntax x))))))
    (n '1)))
```

the expansion of `(n '1)` is `(lambda x ('+ '1 x))`. If the last `x` simply carried an expand-time environment from its source `(syntax x)` expression, then `x` would refer to the outermost `x` binding instead of the one bound by the new `lambda` in the expansion of `(n '1)`.

The difference between the `(n '1)` and `(m)` examples is that `(m)` introduces `x` *after* the `lambda` that should bind `x` has been expanded, while `(n '1)` introduces `x` *before* the `lambda` that should bind `x` is expanded. More generally, `lambda` and `let-syntax` forms can nest arbitrarily, and macros can expand to definitions of macros, so that identifier

bindings and introductions can be interleaved arbitrarily. This combination of *local macros* and *macro-generating macros* defeats a simple capturing of the compile-time environment to bind macro-introduced identifiers.

We can more easily account for identifier binding by renaming identifiers in a syntax object, instead of trying to delay the substitution through an environment. That is, whenever the expander encounters a core binding form like `lambda`, it applies a renaming to the syntax object, instead of merely recording the binding in the expand-time environment. When the expander encounters the first `lambda` in the example containing `(m)`, it renames the binding to $\texttt{x}_1$:

```
((lambda x1
  (let-syntax m (lambda stx (syntax x1))
    (lambda x1
      ('+ (m) x1))))
 1)
```

The macro binding `m` is similarly renamed to $\texttt{m}_1$. When the expander later encounters the inner `lambda`, it renames $\texttt{x}_1$ further to $\texttt{x}_2$.

```
(lambda x2
  ('+ (m1) x2))
```

Since $\texttt{x}_1$ is renamed to $\texttt{x}_2$ only within the inner `lambda` form, then $\texttt{(m}_1\texttt{)}$ expands to a use of $\texttt{x}_1$, which still refers to the outer `lambda` binding.

The example with `(n '1)` works similarly, where the outer `lambda`'s binding is renamed to $\texttt{x}_1$, along with all instances of `x` quoted as syntax in the `n` transformer. The expansion of $\texttt{(n}_1\texttt{ '1)}$ is then $\texttt{(lambda x}_1\texttt{ ('+ '1 x}_1\texttt{))}$, so that the macro-introduced $\texttt{x}_1$ is bound by the macro-introduced binding of $\texttt{x}_1$—both of which will be immediately renamed by the expander to $\texttt{x}_2$.

Renaming is a step in the right direction, but it turns out to be only half of the story. Consider a variation of the `(n '1)` example with `x` in place of `'1`:

```
(lambda x
  (let-syntax n ....
    (n x)))
```

The `x` in `(n x)` should refer to the outer `lambda` binding. According to our story so far, renaming leads to $\texttt{(n}_1\texttt{ x}_1\texttt{)}$, which expands to $\texttt{(lambda x}_1\texttt{ ('+ x}_1\texttt{ x}_1\texttt{))}$, at which point the $\texttt{x}_1$ from $\texttt{(n}_1\texttt{ x}_1\texttt{)}$ is inappropriately captured by the macro-introduced binding of $\texttt{x}_1$.

To avoid this kind of incorrect capture, Dybvig et al. (1993) build on the technique of Kohlbecker et al. (1986). The key is to track syntax objects that are newly introduced by a macro expansion versus syntax objects that were originally provided to the macro expansion. Specifically, the result of a macro transformers is *marked* in such a way that a mark sticks to parts of the expansion that were introduced by the macro, while parts that were present in the macro use are unmarked. Representing marks as superscripts, the expansion of $\texttt{(n}_1\texttt{ x}_1\texttt{)}$ becomes $\texttt{(lambda}^2\texttt{ x}_1{}^2\texttt{ ('+}^2\texttt{ x}_1\texttt{ x}_1{}^2\texttt{))}$, since the `lambda`,

binding $\texttt{x}_1$, `'+`, and last $\texttt{x}_1$ are all introduced by the macro expansion, while the next-to-last $\texttt{x}_1$ was present in the use $\texttt{(n}_1\ \texttt{x}_1\texttt{)}$.

Marks, as represented by subscripts, are not a part of a name in the same way as renamings, a represented by subscripts. In particular, marks do not affect lookup in an expand-time environment, so $\texttt{lambda}^2$ indicates FUN in the same way as `lambda`. Marks affect renamings, however, because a renaming applies only to identifier uses that have the same current name and marks as the binding identifier. Thus, when the expander encounters $\texttt{(lambda}^2\ \texttt{x}_1{}^2\ \texttt{('+ x}_1\ \texttt{x}_1{}^2\texttt{))}$ it renames $\texttt{x}_1{}^2$ to $\texttt{x}_2$, leaving the unmarked $\texttt{x}_1$ alone, so that the result is correctly $\texttt{(lambda x}_2\ \texttt{('+ x}_1\ \texttt{x}_2\texttt{))}$.

3.6.2 Marks and Renames as Lexical Context

In the model, instead of subscripts and superscripts, marks and renames are attached to a syntax object through the lexical-context part of a syntax object. Renames are *not* implemented by changing the symbol within an identifier, because the original symbol is needed if the identifier turns out to be quoted. For example, in

```
(lambda x
  'x)
```

the expander renames `x` to $\texttt{x}_1$, but the body of the `lambda` form should produce the symbol `'x`, not the symbol $\texttt{'x}_1$. Meanwhile, the expander cannot simply skip `quote` forms when renaming, because some quoted forms may not become apparent until macro expansion is complete. By putting renaming information into the lexical-context part of an identifier, the original symbol is intact for quoting.

To support mark and rename information in lexical context, we add two productions to the grammar of *ctx*:

ctx ::= • | (MARK *ctx mrk*) | (RENAME *ctx id name*)
mrk ::= *name*

The MARK and RENAME constructors each build on an existing context. A MARK adds a fresh mark, where a mark is implemented as a name, although integers would work just as well. A RENAME record maps a particular identifier (with its own renamings and marks intact) to a fresh name.

The mark and rename meta-functions push MARK and RENAME records down to all *ctx* chains in a syntax object:

mark⟦(**STX** *atom ctx*), *mrk*⟧ = (**STX** *atom* (MARK *ctx mrk*))
mark⟦(**STX** (**LIST** *stx* ...) *ctx*), *mrk*⟧ = (**STX** (**LIST** mark⟦*stx*, *mrk*⟧ ...)
(MARK *ctx mrk*))
rename⟦(**STX** *atom ctx*), *id*, *name*⟧ = (**STX** *atom* (RENAME *ctx id name*))
rename⟦(**STX** (**LIST** *stx* ...) *ctx*), *id*, *name*⟧ = (**STX** (**LIST** rename⟦*stx*, *id*, *name*⟧ ...)
(RENAME *ctx id name*))

When a transformer expands a macro use, only syntax objects that were introduced by the macro should be marked, while syntax objects that were part of the macro use should remain unmarked. The technique of Dybvig et al. (1993) is to mark the input of a macro transformer using a fresh key, mark the result of the transformer again with the same key,

and treat double marks as canceling each other. This canceling behavior is reflected in the marksof meta-function, which extracts the set of non-canceled marks from an identifier:

$$\begin{array}{l}
\mathsf{marksof}[\![(\textbf{STX}\ \mathit{sym}\ \bullet)]\!] = () \\
\mathsf{marksof}[\![(\textbf{STX}\ \mathit{sym}\ (\textsc{Mark}\ \mathit{ctx}\ \mathit{mrk}))]\!] = \mathit{mrk} \oplus (\mathit{mrk}_2\ ...) \\
\quad \text{where } (\mathit{mrk}_2\ ...) = \mathsf{marksof}[\![(\textbf{STX}\ \mathit{sym}\ \mathit{ctx})]\!] \\
\mathsf{marksof}[\![(\textbf{STX}\ \mathit{sym}\ (\textsc{Rename}\ \mathit{ctx}\ \mathit{id}_2\ \mathit{name}_2))]\!] = \mathsf{marksof}[\![(\textbf{STX}\ \mathit{sym}\ \mathit{ctx})]\!] \\
\\
\mathit{mrk}_1 \oplus (\mathit{mrk}_1\ \mathit{mrk}_2\ ...) = (\mathit{mrk}_2\ ...) \\
\mathit{mrk}_1 \oplus (\mathit{mrk}_2\ ...) = (\mathit{mrk}_1\ \mathit{mrk}_2\ ...)
\end{array}$$

Finally, the redefined resolve meta-function traverses a *ctx* to interpret marks and renamings. The crucial clause in resolve handles a RENAME record, which renames if the source identifier of the rename is consistent with the resolution of the rest of the *ctx*. The two are consistent when they correspond to the same name after nested renamings and when they have the same set of marks:

$$\begin{array}{l}
\mathsf{resolve}[\![(\textbf{STX}\ '\mathit{name}\ \bullet)]\!] = \mathit{name} \\
\mathsf{resolve}[\![(\textbf{STX}\ '\mathit{name}\ (\textsc{Mark}\ \mathit{ctx}\ \mathit{mrk}))]\!] = \mathsf{resolve}[\![(\textbf{STX}\ '\mathit{name}\ \mathit{ctx})]\!] \\
\mathsf{resolve}[\![(\textbf{STX}\ '\mathit{name}\ (\textsc{Rename}\ \mathit{ctx}\ \mathit{id}\ \mathit{name}_{new}))]\!] = \mathit{name}_{new} \\
\quad \text{where } \mathit{name}_1 = \mathsf{resolve}[\![\mathit{id}]\!],\ \mathit{name}_1 = \mathsf{resolve}[\![(\textbf{STX}\ '\mathit{name}\ \mathit{ctx})]\!], \\
\qquad \mathsf{marksof}[\![\mathit{id}]\!] = \mathsf{marksof}[\![(\textbf{STX}\ '\mathit{name}\ \mathit{ctx})]\!] \\
\mathsf{resolve}[\![(\textbf{STX}\ '\mathit{name}\ (\textsc{Rename}\ \mathit{ctx}\ \mathit{id}\ \mathit{name}_2))]\!] = \mathsf{resolve}[\![(\textbf{STX}\ '\mathit{name}\ \mathit{ctx})]\!]
\end{array}$$

The resolve function otherwise ignores marks, which is why a macro-introduced but never renamed `lambda`[2],

$$(\textbf{STX}\ '\texttt{lambda}\ (\textsc{Mark}\ \bullet\ \mathit{mrk}_2))$$

resolves the same as a plain `lambda`,

$$(\textbf{STX}\ '\texttt{lambda}\ \bullet)$$

Note that in the first RENAME case of resolve, when both *id* and (STX *name ctx*) resolve to name_1, and when name_1 is itself the result of renaming, then *id* and (STX *name ctx*) must have the same marks after the renaming—or else the renaming to name_1 would not apply. (Since the expander generates a fresh name for each renaming, any renaming to name_1 will be the same renaming, and hence it requires the same marks wherever it applies.) We can exploit this fact to implement a shortcut in marksof: if a renaming to a given name_1 is encountered, then ignore any remaining marks, because the results for both identifiers will be the same.

To support the shortcut, a revised marksof accepts a traversal-stopping name, the last case of marksof is split into matching and non-matching cases for the name, and the third case of resolve changes to provide the name to marksof:

$$\begin{array}{l}
\mathsf{marksof}[\![(\textbf{STX}\ \mathit{sym}\ \bullet), \mathit{name}]\!] = () \\
\mathsf{marksof}[\![(\textbf{STX}\ \mathit{sym}\ (\textsc{Mark}\ \mathit{ctx}\ \mathit{mrk})), \mathit{name}]\!] = \mathit{mrk} \oplus (\mathit{mrk}_2\ ...) \\
\quad \text{where } (\mathit{mrk}_2\ ...) = \mathsf{marksof}[\![(\textbf{STX}\ \mathit{sym}\ \mathit{ctx}), \mathit{name}]\!] \\
\mathsf{marksof}[\![(\textbf{STX}\ \mathit{sym}\ (\textsc{Rename}\ \mathit{ctx}\ \mathit{id}_2\ \mathit{name})), \mathit{name}]\!] = () \\
\mathsf{marksof}[\![(\textbf{STX}\ \mathit{sym}\ (\textsc{Rename}\ \mathit{ctx}\ \mathit{id}_2\ \mathit{name}_2)), \mathit{name}]\!] = \mathsf{marksof}[\![(\textbf{STX}\ \mathit{sym}\ \mathit{ctx}), \mathit{name}]\!]
\end{array}$$

```
...
resolve⟦(Stx 'name (Rename ctx id name_new))⟧ = name_new
 where name_1 = resolve⟦id⟧,
       name_1 = resolve⟦(Stx 'name ctx)⟧,
       marksof⟦id, name_1⟧ = marksof⟦(Stx 'name ctx), name_1⟧
...
```

As it turns out, this shortcut particularly simplifies the implementation definition contexts, as explained later in Section 3.8.

3.6.3 Adapting the Expander

With the machinery of marks and renames in place, we can adapt our `defmacro`-style macro model to a Scheme-style model by changing the macro-application, `lambda`, and `let-syntax` cases of expand.

A revised macro-application case for expand shows the before-and-after marking operations that track which parts of a syntax object are introduced by a macro expansion:

```
expand⟦stx_macapp, ξ⟧ = expand⟦mark⟦stx_exp, mrk_new⟧, ξ⟧
 where (Stx (List id_mac stx_arg ...) ctx) = stx_macapp,
       val = ξ(resolve⟦id_mac⟧), mrk_new = fresh,
       stx_exp = eval⟦(App val mark⟦stx_macapp, mrk_new⟧)⟧
```

The revised `lambda` case generates a renaming for the formal argument of the function, and then it uses rename to apply the renaming to the body of the `lambda` form:

```
expand⟦(Stx (List id_lam id_arg stx_body) ctx), ξ⟧ = (Stx (List id_lam id_new stx_expbody) ctx)
 where Fun = ξ(resolve⟦id_lam⟧), name_new = fresh,
       id_new = rename⟦id_arg, id_arg, name_new⟧, ξ_new = ξ+{name_new→(Var id_new)},
       stx_expbody = expand⟦rename⟦stx_body, id_arg, name_new⟧, ξ_new⟧
```

The environment is still extended to record that the generated name corresponds to a variable. More generally, the model uses lexical context information to represent the *identity* of bindings, but the compile-time environment still represents the *meanings* of bindings.

Since `let-syntax` introduces a local binding in the same sense as `lambda`, it must rename the local variable in the same way:

```
expand⟦(Stx (List id_ls id stx_rhs stx_body) ctx), ξ⟧ = expand⟦rename⟦stx_body, id, name_new⟧, ξ_new⟧
 where Let-Syntax = ξ(resolve⟦id_ls⟧), name_new = fresh,
       ξ_new = ξ+{name_new→eval⟦parse⟦stx_rhs⟧⟧}
```

With the new expand cases, the `thunk` example of the previous section expands with proper handling of lexical scope:

expand⟦(lambda a (let-syntax
thunk (lambda e .... (**Stx** a •) ....)
(thunk ('+ (**Stx** a •) '1)))),
ξ_0⟧
= (lambda (**Stx** a (Rename • a a2))
expand⟦(let-syntax
thunk (lambda e .... (**Stx** a (Rename • a a2)) ....)
(thunk ('+ (**Stx** a (Rename • a a2)) '1))),
$\xi_1 = \xi_0$+{a2→Var}⟧)
= ... *evaluating the* thunk *binding*...
= (lambda (**Stx** a (Rename • a a2))
expand⟦(thunk ('+ (**Stx** a (Rename • a a2)) '1)),
$\xi_2 = \xi_1$+{thunk→....}⟧)
= ... *calling the* thunk *transformer*...
the macro-introduced a *has the marked context ctx*
= (lambda (**Stx** a (Rename • a a2))
expand⟦(lambda (**Stx** a *ctx* = (Mark (Rename • a a2) mrk_1))
('+ (**Stx** a (Rename • a a2)) '1)),
ξ_2⟧)
and the rename to a3 *applies to* a *with marked context ctx*
= (lambda (**Stx** a (Rename • a a2))
(lambda (**Stx** a (Rename *ctx* (**Stx** a *ctx*) a3))
expand⟦('+ id_{body} '1), $\xi_3 = \xi_2$+{a3→Var}⟧))
where id_{body} = (**Stx** a (Rename (Rename • a a2) (**Stx** a *ctx*) a3))
= ... *expanding the body*...
= (lambda (**Stx** a (Rename • a a2))
(lambda (**Stx** a (Rename *ctx* (**Stx** a *ctx*) a3))
('+ expand⟦id_{body}, ξ_3⟧ '1)))
where the a3 *renaming does not apply, since* id_{body} *is not marked*
= (lambda (**Stx** a (Rename • a a2))
(lambda (**Stx** a (Rename *ctx* (**Stx** a *ctx*) a3))
('+ (**Stx** a (Rename • a a2)) '1)))

3.7 Compile-Time Bindings and Local Expansion

At this point, our model covers macros as they are available in many Scheme implementations. We now add two new primitives that reflect the expanded macro API of Racket: **lvalue** (short for syntax-local-value) for accessing arbitrary compile-time bindings, and **lexpand** (short for local-expand) for forcing the expansion of a sub-form.

The new primitives are available only during the application of a macro transformer, so we add them to a new set of atoms *tprim*:

atom ::= *sym* | *prim* | *tprim* | ....
tprim ::= **lvalue** | **lexpand**

Evaluation of a *tprim* application does not use δ, because it relies on the expansion context. In particular, application of **lvalue** extracts a value from the compile-time environment, and **lexpand** must cancel any mark introduced for the current expansion before starting a nested expansion. We therefore revise eval to accept a compile-time environment and mark in addition to the expression to evaluate.

To evaluate a use of **`lvalue`**, the argument expression is evaluated and must produce an identifier, and the identifier must be mapped to a value in the current compile-time environment, in which case that value is the result of the **`lvalue`** call:

$$\begin{array}{l}\mathsf{eval}[\![(\textbf{APP}\ \texttt{lvalue}\ \mathit{ast}), \xi, \mathit{mrk}]\!] = \xi(\mathsf{resolve}[\![\mathit{id}_{\mathit{result}}]\!]) \\ \quad \text{where } \mathit{id}_{\mathit{result}} = \mathsf{eval}[\![\mathit{ast}, \xi, \mathit{mrk}]\!]\end{array}$$

The essence of **`lexpand`** is that eval for an application of **`lexpand`** must use expand. In addition, **`lexpand`** require two bookkeeping steps:

- Before forcing expansion of the given syntax object, **`lexpand`** applies a mark to cancel the one from the enclosing macro application, and then it adds the mark back after nested expansion (to be canceled again when the enclosing expansion completes). By removing and restoring the mark for an outer expansion that is in progress, **`lexpand`** avoids interference between the original expansion and the sub-form expansion.
- To enable partial expansion, the stop list provided to **`lexpand`** creates new bindings in the compile-time environment to a STOP transformer. In addition, in much the same way that **`lexpand`** removes the current expansion's mark before starting a sub-form expansion, existing strop transformers are removed from the compile-time environment by using nostops, in case a macro transformer that is invoked via **`lexpand`** itself calls **`lexpand`**:

$$\begin{array}{l}\mathsf{nostops}[\![\xi]\!] = \{\mathit{var} \rightarrow \mathit{transformer} \mid \xi(\mathit{var}) = \mathit{transformer} \\ \qquad\qquad\qquad\qquad\qquad\qquad \text{and } \mathit{transformer} \neq \textsc{Stop}\}\end{array}$$

The eval rule for **`lexpand`** puts these steps together, along with evaluation of the arguments to **`lexpand`**:

$$\begin{array}{l}\mathsf{eval}[\![(\textbf{APP}\ \textbf{lexpand}\ \mathit{ast}_{\mathit{expr}}\ \mathit{ast}_{\mathit{stops}}), \xi, \mathit{mrk}]\!] = \mathsf{mark}[\![\mathsf{expand}[\![\mathsf{mark}[\![\mathit{stx}, \mathit{mrk}]\!], \xi_{\mathit{stops}}]\!], \mathit{mrk}]\!] \\ \quad \text{where } \mathit{stx} = \mathsf{eval}[\![\mathit{ast}_{\mathit{expr}}, \xi, \mathit{mrk}]\!], (\textbf{LIST}\ \mathit{id}_{\mathit{stop}}\ ...) = \mathsf{eval}[\![\mathit{ast}_{\mathit{stops}}, \xi, \mathit{mrk}]\!], \\ \qquad \xi_{\mathit{stops}} = \mathsf{nostops}[\![\xi]\!] + \{\mathsf{resolve}[\![\mathit{id}_{\mathit{stop}}]\!] \rightarrow \textsc{Stop}\}\ ...\end{array}$$

Expansion of a form that has a STOP transformer is the same as for a QUOTE transformer, except that multiple sub-forms are allowed inside the form:

$$\begin{array}{l}\mathsf{expand}[\![(\textbf{STX}\ (\textbf{LIST}\ \mathit{id}_{\mathit{stop}}\ \mathit{stx}\ ...)\ \mathit{ctx}), \xi]\!] = (\textbf{STX}\ (\textbf{LIST}\ \mathit{id}_{\mathit{stop}}\ \mathit{stx}\ ...)\ \mathit{ctx}) \\ \quad \text{where } \textsc{Stop} = \xi(\mathsf{resolve}[\![\mathit{id}_{\mathit{stop}}]\!])\end{array}$$

To illustrate, the program

```
(let-syntax
  public (lambda e ('syntax-error))
 (let-syntax
   class (lambda e
           ((lambda e2
              ('car ('cdr ('stx-e e2))))
            ('lexpand ('car ('cdr ('stx-e e)))
                      ('list (syntax public)))))
   (class (public '8))))
```

simulates how `public` in the class system makes sense only within a `class` form (otherwise it reports a syntax error), while `class` locally expands its body stopping at `public` forms. The program expands to `8` roughly as follows (omitting lexical-context information, since it is not directly relevant to the example):

```
  expand⟦(let-syntax public ....
            (let-syntax class .... (class (public '8))))
          ξ0⟧
= ... evaluate transformer expression for public...
= expand⟦(let-syntax class .... (class (public '8))),
          ξ1 = ξ0+{public→....}⟧
= ... evaluate transformer expression for class...
= expand⟦(class (public '8)), ξ2 = ξ1+{class→....}⟧
= ... apply the class transformer...
= eval⟦...., ('lexpand (syntax (public '8))
                       ('list (syntax public))), ....,
      ξ2, mrk1⟧
  expansion stops immediately at public:
= eval⟦...., (syntax (public '8)), ...., ξ2, mrk1⟧
= ... transformer strips the public form away...
= eval⟦(syntax '8), ξ2, mrk1⟧
= expand⟦'8, ξ2⟧
```

Local expansion is consistent with full expansion only when the stop list is either empty or when the stop list contains at least the primitive binding forms. If a stop list omits a binding form, but it includes a form that can wrap a reference to a bound variable, then a partial expansion can produce a different result than full expansion. This effect is illustrated in the following example:

```
(let-syntax
  stop (lambda e ('car ('cdr ('stx-e e))))
 (let-syntax
   ex (lambda e
        ('lexpand ('car ('cdr ('stx-e e)))
                  ('list (syntax stop))))
  (ex (lambda x (let-syntax
                  arg (lambda e (syntax (stop x)))
                 (arg))))))
```

When the `ex` macro forces the expansion of `(lambda x ....)` and stops at uses of `stop`, the result is essentially `(lambda x2 (stop x2))`, where both `x2`s are really `x`s with RENAME wrappers to redirect `x` to some `x2`. The latter `x`, however, also has a MARK due to introduction by the local `arg` macro. Re-expanding `(lambda x2 (stop x2))` therefore produces `(lambda x3 (stop x2))`; since the marks on the two `x2`s are not the same, the new `x3` binding does not capture the inner `x2`.

3.8 Definition Contexts

To support definition contexts, we add two new expansion-time primitives: **`new-defs`** (short for `syntax-local-make-definition-context`) for creating new contexts,

and **def-bind** (short for `syntax-local-bind-syntaxes`) for binding names in a context.

$$\textit{tprim} ::= .... \mid \textbf{new-defs} \mid \textbf{def-bind}$$

A definition context is similar to a RENAME record in a syntax object, except that the set of renamings associated with the context is extensible imperatively. Updates of a definition context require a definition-context store Σ with addresses σ. A DEFS wrapper for syntax objects encapsulates a σ, and DEFS can also tag a σ to form a value. Within Σ, σ maps identifiers to renamed variables.

$$\begin{array}{l}\textit{val} ::= .... \mid (\text{DEFS } \sigma)\\ \textit{ctx} ::= .... \mid (\text{RENAME } \textit{ctx id name } \sigma) \mid (\text{DEFS } \textit{ctx } \sigma)\end{array}$$

$$\begin{array}{l}\sigma ::= \textit{addr} \mid \text{NULL}\\ \Sigma ::= \text{definition-context store}, \sigma \rightarrow (\textit{id} \rightarrow \textit{sym})\\ S ::= \text{set of } \sigma\end{array}$$

When resolve encounters a DEFS wrapper, it unpacks the wrapper into a sequence of RENAME wrappers. For reasons explained below, the generated RENAME wrappers must record the source definition context. Thus, RENAME is extended above to include an address σ. The special address NULL is used for RENAME wrappers that originate from `lambda` or `let-syntax` renamings.

The expand cases must be revised to take a Σ argument and produce a resulting $\langle \textit{stx}, \Sigma\rangle$ tuple, and eval must similarly consume and produce a Σ. For the existing cases, the Σ is simply carried through, including to the resolve and parse meta-functions. The new case in eval for a **new-defs** form, however, extends the definition-context store with a new, empty definition context:

$$\begin{array}{l}\mathsf{eval}[\![(\textbf{APP } \textbf{new-defs}), \xi, \textit{mrk}, \Sigma]\!] = \langle(\text{DEFS } \sigma), \xi, \Sigma{+}\{\sigma{\rightarrow}\varnothing\}\rangle\\ \quad \text{where } \sigma = \text{fresh}\end{array}$$

A new eval case handles the use of **def-bind** to extend a definition context with an identifier that corresponds to a run-time binding. A run-time binding is just like one created by `lambda`, and **def-bind** similarly generates a fresh variable and maps the original identifier to the new variable for syntax objects. While the expansion of `lambda` applies the renaming to a body expression, evaluation of **def-bind** records the renaming in a definition context. Evaluation of **def-bind** also extends the compile-time environment to indicate that the generated variable maps to itself, just like the expansion of `lambda`.

$$\begin{array}{l}\mathsf{eval}[\![(\textbf{APP } \textbf{def-bind } \textit{ast}_{\textit{defs}}\ \textit{ast}_{\textit{id}}), \xi, \textit{mrk}, \Sigma]\!] = \langle 0, \xi_2{+}\{\textit{name}_{\textit{new}}{\rightarrow}(\text{VAR } \textit{id}_{\textit{new}})\}, \Sigma_3\rangle\\ \quad \text{where } \langle(\text{DEFS } \sigma), \xi_1, \Sigma_1\rangle = \mathsf{eval}[\![\textit{ast}_{\textit{defs}}, \xi, \textit{mrk}, \Sigma]\!],\\ \qquad \langle \textit{id}, \xi_2, \Sigma_2\rangle = \mathsf{eval}[\![\textit{ast}_{\textit{id}}, \xi_1, \textit{mrk}, \Sigma_1]\!], \textit{name}_{\textit{new}} = \text{fresh},\\ \qquad \textit{id}_{\textit{new}} = \mathsf{rename}[\![\textit{id}, \textit{id}, \textit{name}_{\textit{new}}]\!],\\ \qquad \Sigma_3 = \Sigma_2{+}\{\sigma{\rightarrow}\Sigma_2(\sigma){+}\{\mathsf{mark}[\![\textit{id}, \textit{mrk}]\!]{\rightarrow}\textit{name}_{\textit{new}}\}\}\end{array}$$

When **def-bind** is used to bind an identifier to a compile-time value (including a macro transformer), the given compile-time expression must be evaluated, and then its result can be bound in the environment. Like the evaluation case for binding variables, the case for binding a compile-time value generates a fresh variable, maps it in the definition context, and extends the compile-time environment. In this case, however, the extended compile-time environment contains a compile-time value, instead of just a variable.

$$\begin{array}{l}
\mathsf{eval}[\![(\textbf{APP}\ \texttt{\textbf{def-bind}}\ \mathit{ast}_{\mathit{defs}}\ \mathit{ast}_{\mathit{id}}\ \mathit{ast}_{\mathit{stx}}), \xi, \mathit{mrk}, \Sigma]\!] = \langle 0, \xi_4{+}\{\mathit{name}_{\mathit{new}} \rightarrow \mathit{val}\}, \Sigma_5\rangle \\
\quad \text{where } \langle(\textsc{Defs}\ \sigma), \xi_1, \Sigma_1\rangle = \mathsf{eval}[\![\mathit{ast}_{\mathit{defs}}, \xi, \mathit{mrk}, \Sigma]\!], \\
\qquad \langle \mathit{id}, \xi_2, \Sigma_2\rangle = \mathsf{eval}[\![\mathit{ast}_{\mathit{id}}, \xi_1, \mathit{mrk}, \Sigma_1]\!], \\
\qquad \langle \mathit{stx}, \xi_3, \Sigma_3\rangle = \mathsf{eval}[\![\mathit{ast}_{\mathit{stx}}, \xi_2, \mathit{mrk}, \Sigma_2]\!], \\
\qquad \langle \mathit{val}, \xi_4, \Sigma_4\rangle = \mathsf{eval}[\![\mathsf{parse}[\![\mathsf{defs}[\![\mathsf{mark}[\![\mathit{stx}, \mathit{mrk}]\!], \sigma]\!], \Sigma_3]\!], \xi_3, \mathit{mrk}, \Sigma_3]\!], \\
\qquad \mathit{name}_{\mathit{new}} = \mathsf{fresh}, \mathit{id}_{\mathit{new}} = \mathsf{rename}[\![\mathit{id}, \mathit{id}, \mathit{name}_{\mathit{new}}]\!], \\
\qquad \Sigma_5 = \Sigma_4{+}\{\sigma \rightarrow \Sigma_4(\sigma){+}\{\mathsf{mark}[\![\mathit{id}, \mathit{mrk}]\!] \rightarrow \mathit{name}_{\mathit{new}}\}\}
\end{array}$$

A definition context is associated with an expression by extending **`lexpand`** to accept a definition context as its last argument. The definition context is applied to the given expression before it is expanded (using a defs meta-function that is like rename and mark), so that expansion uses the context. Less obviously, the definition context is also added to the result of local expansion. For syntax objects introduced by local expansion, the second addition ensures that if the introduced syntax objects correspond to a definition, the definition's binding will use the correct lexical context.

$$\begin{array}{l}
\mathsf{eval}[\![(\textbf{APP}\ \texttt{\textbf{lexpand}}\ \mathit{ast}_{\mathit{expr}}\ \mathit{ast}_{\mathit{stops}}\ \mathit{ast}_{\mathit{defs}}), \xi, \mathit{mrk}, \Sigma]\!] = \langle \mathsf{mark}[\![\mathsf{defs}[\![\mathit{stx}, \sigma]\!], \mathit{mrk}]\!], \xi_3, \Sigma_4\rangle \\
\quad \text{where } \langle \mathit{stx}_{\mathit{expr}}, \xi_1, \Sigma_1\rangle = \mathsf{eval}[\![\mathit{ast}_{\mathit{expr}}, \xi, \mathit{mrk}, \Sigma]\!], \\
\qquad \langle(\textbf{LIST}\ \mathit{id}_{\mathit{stop}}\ ...), \xi_2, \Sigma_2\rangle = \mathsf{eval}[\![\mathit{ast}_{\mathit{stops}}, \xi_1, \mathit{mrk}, \Sigma_1]\!], \\
\qquad \langle(\textsc{Defs}\ \sigma), \xi_3, \Sigma_3\rangle = \mathsf{eval}[\![\mathit{ast}_{\mathit{defs}}, \xi_2, \mathit{mrk}, \Sigma_2]\!], \\
\qquad \xi_{\mathit{stops}} = \mathsf{nostops}[\![\xi_3]\!]{+}\{\mathsf{resolve}[\![\mathit{id}_{\mathit{stop}}, \Sigma_3]\!] \rightarrow \textsc{Stop}\}\ ..., \\
\qquad \mathit{stx}_{\mathit{new}} = \mathsf{defs}[\![\mathsf{mark}[\![\mathit{stx}_{\mathit{expr}}, \mathit{mrk}]\!], \sigma]\!], \langle \mathit{stx}, \Sigma_4\rangle = \mathsf{expand}[\![\mathit{stx}_{\mathit{new}}, \xi_{\mathit{stops}}, \Sigma_3]\!]
\end{array}$$

Given the extended definition of *ctx*, a natural extension of resolve is to add a DEFS clause while generally extending resolve to accept the current store Σ. The new DEFS clause could simply unpack the wrapper into a set of RENAME wrappers based on the content of the definition context in the store, then recur:

$$\begin{array}{l}
\mathsf{renames}[\![\sigma, (), \mathit{ctx}]\!] = \mathit{ctx} \\
\mathsf{renames}[\![\sigma, ((\mathit{id}\ \mathit{name})\ (\mathit{id}_2\ \mathit{name}_2)\ ...), \mathit{ctx}]\!] = \mathsf{renames}[\![\sigma, ((\mathit{id}_2\ \mathit{name}_2)\ ...), \\
\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad (\textsc{Rename}\ \mathit{ctx}\ \mathit{id}\ \mathit{name}\ \sigma)]\!] \\
\mathsf{resolve}[\![(\textbf{STX}\ \mathit{val}\ (\textsc{Defs}\ \mathit{ctx}\ \sigma)), \Sigma]\!] = \mathsf{resolve}[\![(\textbf{STX}\ \mathit{val}\ \mathit{ctx}_{\mathit{new}}), \Sigma]\!] \\
\quad \text{where } \{\mathit{id} \rightarrow \mathit{name}_{\mathit{new}}\ ...\} = \Sigma(\sigma), \\
\qquad \mathit{ctx}_{\mathit{new}} = \mathsf{renames}[\![\sigma, ((\mathit{id}\ \mathit{name}_{\mathit{new}})\ ...), \mathit{ctx}]\!]
\end{array}$$

This simple extension of resolve does not work, because it does not terminate when σ is part of a cycle in Σ. A cycle is created in Σ for most definition contexts, because each defined identifier is placed into the context where the bindings occur. More complex cycles are created when definition contexts are nested, as in nested `define-package` forms (see Section 2.5). For example, in the Racket expression

```
(define-package p ()
  (define x 1)
  (define-package q ()
    (define x 2)))
```

the two defined identifiers must resolve to different bindings, say, `x1` and `x3`. The corresponding syntax objects are roughly

$(\mathbf{STX}\ \mathtt{x}\ (\text{DEFS}\ (\text{DEFS}\ \bullet\ \sigma_1)\ \sigma_1))$
$(\mathbf{STX}\ \mathtt{x}\ (\text{DEFS}\ (\text{DEFS}\ (\text{DEFS}\ (\text{DEFS}\ \bullet\ \sigma_1)\ \sigma_2)\ \sigma_2)\ \sigma_1))$

$\Sigma = \{\sigma_1 \leftarrow \{(\mathbf{STX}\ \mathtt{x}\ (\text{DEFS}\ \bullet\ \sigma_1)) \leftarrow \mathtt{x1},$
$\quad (\mathbf{STX}\ \mathtt{x}\ (\text{DEFS}\ (\text{DEFS}\ (\text{DEFS}\ (\text{DEFS}\ \bullet\ \sigma_1)\ \sigma_2)\ \sigma_2)\ \sigma_1)) \leftarrow \mathtt{x3}\}$
$\quad \sigma_2 \leftarrow \{(\text{DEFS}\ (\text{DEFS}\ (\text{DEFS}\ \bullet\ \sigma_1)\ \sigma_2)\ \sigma_2) \leftarrow \mathtt{x2}\}\}$

The first identifier has σ_1 twice, because the identifier appears in both the first `define` form and its expansion. The second identifier has nested σs, because it appears before and after both the outer expansion of `define-package` and the inner expansion of `define`. In Σ, the σ_1 binding reflects the final identifiers. The σ_2 binding reflects the state of the second `x` by the time it was bound for the inner package. It was put into the σ_1 context during the expansion of the `define-package` form, and then put into the σ_2 context before and after expanding the inner `define`. The inner expansion created the temporary binding `x2`, but it was later subsumed by the `x3` binding for the enclosing context.

To accommodate cycles within Σ, resolve must keep track of which contexts are already being used toward a renaming. For an initial call to resolve, no contexts are already being used. When a DEFS tag is unpacked into RENAMEs, then the corresponding context is already being used for the purposes of checking targets of renamings in the branches of the wrapper (i.e., the *id* in each RENAME wrapper). However, the context is not yet used for the spine of the lexical-context wrapper, because if no renaming applies among the unpacked ones, a later DEFS wrapper for the same definition context might apply. Thus, resolve accepts two sets of definition contexts to be skipped: one for the spine, and one for branches that are rename targets.

$\mathsf{resolve}[\![stx, \Sigma]\!] = \mathsf{resolve}^*[\![stx, \Sigma, \varnothing, \varnothing]\!]$

....
$\mathsf{resolve}^*[\![(\mathbf{STX}\ 'name\ (\text{RENAME}\ ctx\ id_{orig}\ name_{new}\ \sigma)), \Sigma, S_{spine}, S_{branch}]\!] = name_{new}$
where $name_1 = \mathsf{resolve}^*[\![id_{orig}, \Sigma, S_{branch}, S_{branch}]\!]$,
$name_1 = \mathsf{resolve}^*[\![(\mathbf{STX}\ 'name\ ctx), \Sigma, \{\sigma\} \cup S_{spine}, S_{branch}]\!]$,
$\mathsf{marksof}[\![id_{orig}, name_1]\!] = \mathsf{marksof}[\![(\mathbf{STX}\ 'name\ ctx), name_1]\!]$
$\mathsf{resolve}^*[\![(\mathbf{STX}\ 'name\ (\text{DEFS}\ ctx\ \sigma)), \Sigma, S_{spine}, S_{branch}]\!] = \mathsf{resolve}^*[\![(\mathbf{STX}\ 'name\ ctx), \Sigma, S_{spine}, S_{branch}]\!]$
where $\sigma \in S_{spine}$
$\mathsf{resolve}^*[\![(\mathbf{STX}\ 'name\ (\text{DEFS}\ ctx\ \sigma)), \Sigma, S_{spine}, S_{branch}]\!] = \mathsf{resolve}^*[\![(\mathbf{STX}\ 'name\ ctx_{new}), \Sigma, S_{spine}, \{\sigma\} \cup S_{branch}]\!]$
where $\{id \rightarrow name_{new}\ ...\} = \Sigma(\sigma)$, $ctx_{new} = \mathsf{renames}[\![\sigma, ((id\ name_{new})\ ...), ctx]\!]$

Finally, we revise marksof to handle DEFS wrappers, taking care to properly support renaming of identifiers that are bound in a definition context. Consider the following Racket example:

```
(lambda ()
  (define x 1)
  (define-syntax m (lambda (stx) #'(list x)))
  (m))
```

When a definition context is used to expand the body forms of this `lambda`, then all identifiers acquire the definition context. The local expansion of `(m)`, furthermore, produces

an `x` that has a mark in addition to the definition context. If the `x` from `define` is then used as a `letrec` binding to continue expansion, then the extra mark on the `x` from `(m)` could prevent it from being bound by the `letrec`. This is the same potential problem as described at the end of Section 3.7, and it occurs because `(m)` is expanded only far enough to discover that it acts an expression rather than a definition.

To avoid this problem, resolve must ignore marks that are introduced during partial expansion for identifiers that are bound by the partial expansion's definition context. Ignoring such marks simulates a complete expansion, which would replace a marked variable with the fresh name that is used for its binding. Since **`lexpand`** adds the definition context to both its argument and its result, two instances of the definition context serve to bracket the marks that should be ignored later by resolve. Combining this idea with the observation from Section 3.6.2 that marks *after* the definition-context renaming can be ignored for a further renaming, it suffices to make marksof ignore all marks after the first instance of a definition-context renaming:

$$\begin{aligned}
&\mathsf{marksof}[\![(\textbf{STX}\ \mathit{val}\ (\textsc{Defs}\ \mathit{ctx}\ \sigma)), \Sigma, \mathit{name}]\!] = ()\\
&\quad \text{where } \mathit{name} \in \mathrm{rng}(\Sigma(\sigma))\\
&\mathsf{marksof}[\![(\textbf{STX}\ \mathit{val}\ (\textsc{Defs}\ \mathit{ctx}\ \sigma)), \Sigma, \mathit{name}]\!] = \mathsf{marksof}[\![(\textbf{STX}\ \mathit{val}\ \mathit{ctx}), \Sigma, \mathit{name}]\!]
\end{aligned}$$

4 Related Work

Our model builds directly on the model of Dybvig et al. (1993), adding extensions for compile-time bindings, partial expansion, and definition contexts. Another difference in our model is that the expander, which maps syntax objects to syntax objects, is decoupled from the parser, which maps syntax objects to an executable AST. This change allows us to model `local-expand`, but it also reflects Racket's pervasive use of syntax objects as a basis for program analysis and transformation (see Section 2.6). Previous models of Scheme macros do not account for the handling of internal-definition contexts (e.g., in the body of `lambda`); *ribs*, as described informally by Waddell and Dybvig (1999), sound similar to our definition contexts, but no model is provided, and the construct is not accessible to macro transformers.

Our model inherits one drawback of the Dybvig et al. (1993) model: whether marks and renamings actually make macros respect lexical scope as intended is hardly apparent. Other models of macros face similar problems:

- Gasbichler (2006) attacked the gap between specification and mechanism in his model of macros based on explicit substitutions, but Gasbichler's model treats pattern-variable bindings differently than other bindings, which turns out not to work completely right for macro-generating macros, thus leaving a gap in the explanation.
- The λ_{m} calculus of Herman (2010) creates a tight correspondence between specification and behavior for a restricted subset of `syntax-rules` macros. The λ_{m} calculus uses a custom type system to specify the binding structure of a macro's arguments. Expressions are annotated with the new bindings brought into scope, and macros with ambiguous scoping rules are disallowed. The calculus does not handle the flexibility and power of `syntax-case` macros, and the type system would re-

quire significant extension to represent the essence of local expansion and definition contexts.

- Other frameworks for lexically-scoped macros, notably syntactic closures (Bawden and Rees 1988) and explicit renaming (Clinger 1991), use a notion of lexical context that more directly maps to the programmer's view of binding scopes. Unfortunately, the more direct representation moves binding information into the expansion environment; in the case of syntactic closures, it tangles the representation of syntax and expansion environments, and in the case of explicit renaming, identifier comparisons depend on a compile-time environment. Our goals require a purely "syntactic" representation of syntax, which can be locally expanded, transported into a new context, and then re-expanded.

An important direction for further research is to find a model with the syntactic advantage of Dybvig et al. (1993), but with a more obvious connection to the usual notion of binding scopes, that is able to support our extensions for cooperation among macros.

Previous work on expansion-passing style macros (Dybvig et al. 1988) addresses the problem of expanding sub-forms in a macro use. In expansion-passing style, a macro receives two arguments: the term to transform and an expander function. The macro can call the function to expand sub-forms, and it can pass a modified expander function to be used for the sub-form expansion. Similarly, Common Lisp provides the functions `macroexpand` and `macroexpand-1`, as well as an expansion hook `*macroexpand-hook*`. Both of these mechanisms give macros the power to expand sub-forms, and they give a macro the ability to change the expander's behavior for the duration of the sub-form expansion. In contrast, `local-expand` always invokes the standard expander, allowing only the addition of new stopping conditions and an optional definition context. These restrictions make `local-expand` less powerful but more predictable than previous mechanisms for sub-form expansion. In addition, `local-expand` works with macros that respect lexical scope, whereas previous facilities were developed for scope-oblivious systems.

Continuation-passing style (CPS) also enables a kind of sub-form expansion for macros, as described by Hilsdale and Friedman (2000). Only macros explicitly written in CPS can participate in sub-form expansion, so such macros cannot easily re-use existing forms like `define`. Furthermore, since macros cannot verify that sub-forms follow the protocol, mistakes generally lead to mysterious error messages at best and bewildering behavior at worst.

Compile-time meta-programming in the style of Template Haskell (Sheard and Peyton Jones 2002) supports the expansion of sub-forms within a macro transformer, because macros are compile-time functions that can be called directly from other compile-time functions. Macros in Template Haskell also respect lexical scope. Unlike Lisp and Scheme, however, uses of macros must be explicitly marked in the program source with a leading `$`, which creates different demands on the representation of syntax and the resolution of binding. For example, an identifier's role within a template as binder or not can be determined immediately, whereas the determination must be delayed within Scheme templates. The advantage of Scheme-style macros, and the target of our work, is to allow new syntac-

tic forms that have the same status as built-in syntactic forms, thus supporting a tower of languages.

Other systems address the need for cooperation and communication of language extension at a different level. Ziggurat (Fisher and Shivers 2008) and Silver (Van Wyk et al. 2009) both support static analysis in languages with extensible syntax. Expansion or "delegation" is automatically triggered by the system as necessary to support analyses, and expansion proceeds only far enough to produce a syntactic form that can be analyzed. Compared to Ziggurat and Silver, macro expansion in Racket is simpler and at a lower level; only expansion and binding information is available for a sub-term, and other information must be encoded in the expansion.

Language constructs based on fresh names (Gabbay and Pitts 1999; Shinwell et al. 2003) or higher-order abstract syntax (Pfenning and Elliott 1988; Pfenning and Schürmann 1999) address the problem of manipulating program fragments with bindings, but they have different operations than syntax objects. Programs using fresh-name features explicitly open and close term representations, instead of automatically absorbing lexical information. With higher-order abstract syntax, binders and bindings are implicit, instead of entities that can be manipulated explicitly. Syntax objects fit somewhere in between: lexical information is maintained automatically, but it can be manipulated more directly.

Acknowledgments Thanks to Robert Ransom, Michael Sperber, Mitch Wand, Paul Stansifer, and the anonymous referees for many helpful comments and suggestions.

Bibliography

Alan Bawden and Jonathan Rees. Syntactic Closures. In *Proc. Lisp and Functional Programming*, pp. 86–95, 1988.

William Clinger and Jonathan Rees. Macros that Work. In *Proc. ACM Symp. Principles of Programming Languages*, pp. 155–162, 1991.

William D. Clinger. Hygienic Macros Through Explicit Renaming. *Lisp Pointers* 4(4), pp. 25–28, 1991.

R. Kent Dybvig, Daniel P. Friedman, and Christopher T. Haynes. Expansion-Passing Style: A General Macro Mechanism. *Lisp and Symbolic Computation* 1(1), pp. 53–75, 1988.

R. Kent Dybvig, Robert Hieb, and Carl Bruggeman. Syntactic Abstraction in Scheme. *Lisp and Symbolic Computation* 5(4), pp. 295–326, 1993.

Matthias Felleisen, Robert Bruce Findler, and Matthew Flatt. *Semantics Engineering with PLT Redex*. MIT Press, 2009.

David Fisher and Olin Shivers. Building Language Towers with Ziggurat. *J. Functional Programming* 18(5-6), pp. 707–780, 2008.

Matthew Flatt. Compilable and Composable Macros, You Want it *When?* In *Proc. ACM Intl. Conf. Functional Programming*, pp. 72–83, 2002.

Murdoch Gabbay and Andrew Pitts. A New Approach to Abstract Syntax Involving Binders. In *Proc. IEEE Symp. Logic in Computer Science*, pp. 341–363, 1999.

Martin Gasbichler. Fully-parameterized Higher-order Modules with Hygienic Macros. PhD dissertation, Universität Tübingen, 2006.

Abdulaziz Ghuloum and R. Kent Dybvig. Portable syntax-case. 2007. `https://www.cs.indiana.edu/chezscheme/syntax-case/`

David Herman. A Theory of Typed Hygienic Macros. PhD dissertation, Northeastern University, 2010.

Erik Hilsdale and Daniel P. Friedman. Writing Macros in Continuation-Passing Style. In *Proc. Scheme and Functional Programming*, 2000.

Eugene Kohlbecker, Daniel P. Friedman, Matthias Felleisen, and Bruce Duba. Hygienic Macro Expansion. In *Proc. Lisp and Functional Programming*, pp. 151–181, 1986.

Eugene E. Kohlbecker and Mitchell Wand. Macro-by-Example: Deriving Syntactic Transformations from their Specifications. In *Proc. ACM Symp. Principles of Programming Languages*, pp. 77–84, 1987.

Robert Milner, Mads Tofte, and Robert Harper. *The Definition of Standard ML*. MIT Press, 1990.

Frank Pfenning and Conal Elliott. Higher-order Abstract Syntax. In *Proc. ACM Conf. Programming Language Design and Implementation*, pp. 199–208, 1988.

Frank Pfenning and Carsten Schürmann. System Description: Twelf – A Meta-Logical Framework for Deductive Systems. In *Proc. Intl. Conf. Automated Deduction*, pp. 202–206, 1999.

Tim Sheard and Simon Peyton Jones. Template Meta-programming for Haskell. In *Proc. Haskell Wksp.*, pp. 60–75, 2002.

Mark R. Shinwell, Andrew M. Pitts, and Murdoch J. Gabbay. FreshML: Programming with Binders Made Simple. In *Proc. ACM Intl. Conf. Functional Programming*, pp. 263–274, 2003.

Michael Sperber (Ed.). The Revised ^{6} Report on the Algorithmic Language Scheme. 2007.

André van Tonder. R6RS Libraries and Macros . 2007. `http://www.het.brown.edu/people/andre/macros/`

Erik Van Wyk, Derek Bodin, Lijesh Krishnan, and Jimin Gao. Silver: an Extensible Attribute Grammar System. *Science of Computer Programming*, 2009.

Oscar Waddell and R. Kent Dybvig. Extending the Scope of Syntactic Abstraction. In *Proc. ACM Symp. Principles of Programming Languages*, pp. 203–213, 1999.

Appendix

ast ::= var | (**App** ast ast ...) | val
val ::= (**Fun** var ast) | $atom$
| (**List** val ...) | stx | (Defs σ)
$atom$ ::= sym | $prim$ | $tprim$ | ....
sym ::= '$name$
$prim$ ::= **`stx-e`** | **`mk-stx`** | ....
$tprim$ ::= **`new-defs`** | **`def-bind`** | **`lvalue`** | **`lexpand`**
σ ::= $addr$ | NULL

ξ ::= a mapping from $name$ to $transformer$
Σ ::= definition-context store, $\sigma \to (id \to sym)$
S ::= set of σ

stx ::= (**Stx** $atom$ ctx)
| (**Stx** (**List** stx ...) ctx)
id ::= (**Stx** sym ctx)
ctx ::= •
| (Rename ctx id $name$ σ)
| (Mark ctx mrk)
| (Defs ctx σ)

$transformer$ ::= Fun | Let-Syntax | Quote
| (Var id) | val | Stop

$name$ ::= a token such as `x`, `egg`, or `lambda`
var ::= (**Var** $name$)
$addr$, mrk ::= $name$

δ(**`stx-e`** (**Stx** val ctx)) = val
δ(**`mk-stx`** $atom$ (**Stx** val ctx)) = (**Stx** $atom$ ctx)
δ(**`mk-stx`** (**List** stx ...) (**Stx** val ctx)) = (**Stx** (**List** stx ...) ctx)
...

$\mathsf{eval}[\![(\textbf{App}\ (\textbf{Fun}\ var\ ast_{body})\ ast_{arg}), \xi, mrk, \Sigma]\!] = \mathsf{eval}[\![ast_{body}[var \leftarrow val], \xi_1, mrk, \Sigma_1]\!]$
where $\langle val, \xi_1, \Sigma_1\rangle = \mathsf{eval}[\![ast_{arg}, \xi, mrk, \Sigma]\!]$

$\mathsf{eval}[\![(\textbf{App}\ prim\ ast_{arg}\ ...), \xi, mrk, \Sigma]\!] = \langle \delta(prim\ val\ ...), \xi_1, \Sigma_1\rangle$
where $\langle (val\ ...), \xi_1, \Sigma_1\rangle = \mathsf{eval}^*[\![(), (ast_{arg}\ ...), \xi, mrk, \Sigma]\!]$

$\mathsf{eval}[\![(\textbf{App}\ ast_{op}\ ast_{arg}\ ...)]\!] = \mathsf{eval}[\![(\textbf{App}\ \mathsf{eval}[\![ast_{op}]\!]\ ast_{arg}\ ...)]\!]$

$\mathsf{eval}[\![val, \xi, mrk, \Sigma]\!] = \langle val, \xi, \Sigma\rangle$

$\mathsf{eval}[\![(\textbf{App}\ \textbf{`lvalue`}\ ast), \xi, mrk, \Sigma]\!] = \langle \xi(\mathsf{resolve}[\![stx, \Sigma_1]\!]), \xi_1, \Sigma_1\rangle$
where $\langle stx, \xi_1, \Sigma_1\rangle = \mathsf{eval}[\![ast, \xi, mrk, \Sigma]\!]$

$\mathsf{eval}[\![(\textbf{App}\ \textbf{`lexpand`}\ ast_{expr}\ ast_{stops}\ ast_{defs}), \xi, mrk, \Sigma]\!] = \langle \mathsf{mark}[\![\mathsf{defs}[\![stx, \sigma]\!], mrk]\!], \xi_3, \Sigma_4\rangle$
where $\langle stx_{expr}, \xi_1, \Sigma_1\rangle = \mathsf{eval}[\![ast_{expr}, \xi, mrk, \Sigma]\!]$,
$\langle (\textbf{List}\ id_{stop}\ ...), \xi_2, \Sigma_2\rangle = \mathsf{eval}[\![ast_{stops}, \xi_1, mrk, \Sigma_1]\!]$,
$\langle (\textsc{Defs}\ \sigma), \xi_3, \Sigma_3\rangle = \mathsf{eval}[\![ast_{defs}, \xi_2, mrk, \Sigma_2]\!]$,
$\xi_{stops} = \mathsf{nostops}[\![\xi_3]\!] + \{\mathsf{resolve}[\![id_{stop}, \Sigma_3]\!] \to \textsc{Stop}\}$ ...,
$stx_{new} = \mathsf{defs}[\![\mathsf{mark}[\![stx_{expr}, mrk]\!], \sigma]\!]$, $\langle stx, \Sigma_4\rangle = \mathsf{expand}[\![stx_{new}, \xi_{stops}, \Sigma_3]\!]$

$\mathsf{eval}[\![(\textbf{App}\ \textbf{`new-defs`}), \xi, mrk, \Sigma]\!] = \langle (\textsc{Defs}\ \sigma), \xi, \Sigma + \{\sigma \to \varnothing\}\rangle$
where σ = fresh

$\mathsf{eval}[\![(\textbf{App}\ \textbf{`def-bind`}\ ast_{defs}\ ast_{id}), \xi, mrk, \Sigma]\!] = \langle 0, \xi_2 + \{name_{new} \to (\textsc{Var}\ id_{new})\}, \Sigma_3\rangle$
where $\langle (\textsc{Defs}\ \sigma), \xi_1, \Sigma_1\rangle = \mathsf{eval}[\![ast_{defs}, \xi, mrk, \Sigma]\!]$, $\langle id, \xi_2, \Sigma_2\rangle = \mathsf{eval}[\![ast_{id}, \xi_1, mrk, \Sigma_1]\!]$,
$name_{new}$ = fresh, $id_{new} = \mathsf{rename}[\![id, id, name_{new}]\!]$,
$\Sigma_3 = \Sigma_2 + \{\sigma \to \Sigma_2(\sigma) + \{\mathsf{mark}[\![id, mrk]\!] \to name_{new}\}\}$

$\mathsf{eval}[\![(\textbf{App}\ \textbf{`def-bind`}\ ast_{defs}\ ast_{id}\ ast_{stx}), \xi, mrk, \Sigma]\!] = \langle 0, \xi_4 + \{name_{new} \to val\}, \Sigma_5\rangle$
where $\langle (\textsc{Defs}\ \sigma), \xi_1, \Sigma_1\rangle = \mathsf{eval}[\![ast_{defs}, \xi, mrk, \Sigma]\!]$, $\langle id, \xi_2, \Sigma_2\rangle = \mathsf{eval}[\![ast_{id}, \xi_1, mrk, \Sigma_1]\!]$,
$\langle stx, \xi_3, \Sigma_3\rangle = \mathsf{eval}[\![ast_{stx}, \xi_2, mrk, \Sigma_2]\!]$,
$\langle val, \xi_4, \Sigma_4\rangle = \mathsf{eval}[\![\mathsf{parse}[\![\mathsf{defs}[\![\mathsf{mark}[\![stx, mrk]\!], \sigma]\!], \Sigma_3]\!], \xi_3, mrk, \Sigma_3]\!]$,
$name_{new}$ = fresh, $id_{new} = \mathsf{rename}[\![id, id, name_{new}]\!]$,
$\Sigma_5 = \Sigma_4 + \{\sigma \to \Sigma_4(\sigma) + \{\mathsf{mark}[\![id, mrk]\!] \to name_{new}\}\}$

$\mathsf{eval}^*[\![(val\ ...), (), \xi, mrk, \Sigma]\!] = \langle (val\ ...), \xi, \Sigma\rangle$

$\mathsf{eval}^*[\![(val\ ...), (ast_0\ ast_1\ ...), \xi, mrk, \Sigma]\!] = \mathsf{eval}^*[\![(val\ ...\ val_0), (ast_1\ ...), \xi_1, mrk, \Sigma_1]\!]$
where $\langle val_0, \xi_1, \Sigma_1\rangle = \mathsf{eval}[\![ast_0, \xi, mrk, \Sigma]\!]$

$$\begin{aligned}
&\mathsf{expand}[\![(\textbf{Stx}\ (\textbf{List}\ id_{lam}\ id_{arg}\ stx_{body})\ ctx), \xi, \Sigma]\!] \\
&\quad = \langle(\textbf{Stx}\ (\textbf{List}\ id_{lam}\ id_{new}\ stx_{expbody})\ ctx), \Sigma_1\rangle \\
&\quad \text{where } \textsc{Fun} = \xi(\mathsf{resolve}[\![id_{lam}, \Sigma]\!]),\ name_{new} = \mathsf{fresh},\ id_{new} = \mathsf{rename}[\![id_{arg}, id_{arg}, name_{new}]\!], \\
&\qquad stx_{newbody} = \mathsf{rename}[\![stx_{body}, id_{arg}, name_{new}]\!],\ \xi_{new} = \xi + \{name_{new} \to (\textsc{Var}\ id_{new})\}, \\
&\qquad \langle stx_{expbody}, \Sigma_1\rangle = \mathsf{expand}[\![stx_{newbody}, \xi_{new}, \Sigma]\!] \\
&\mathsf{expand}[\![(\textbf{Stx}\ (\textbf{List}\ id_{quote}\ stx)\ ctx), \xi, \Sigma]\!] \\
&\quad = \langle(\textbf{Stx}\ (\textbf{List}\ id_{quote}\ stx)\ ctx), \Sigma\rangle \\
&\quad \text{where } \textsc{Quote} = \xi(\mathsf{resolve}[\![id_{quote}, \Sigma]\!]) \\
&\mathsf{expand}[\![(\textbf{Stx}\ (\textbf{List}\ id_{ls}\ id_{mac}\ stx_{rhs}\ stx_{body})\ ctx), \xi, \Sigma]\!] \\
&\quad = \mathsf{expand}[\![stx_{newbody}, \xi + \{name_{new} \to val\}, \Sigma_1]\!] \\
&\quad \text{where } \textsc{Let-Syntax} = \xi(\mathsf{resolve}[\![id_{ls}, \Sigma]\!]),\ name_{new} = \mathsf{fresh}, \\
&\qquad \langle val, \xi_1, \Sigma_1\rangle = \mathsf{eval}[\![\mathsf{parse}[\![stx_{rhs}, \Sigma]\!], \xi, \texttt{no-mrk}, \Sigma]\!], \\
&\qquad stx_{newbody} = \mathsf{rename}[\![stx_{body}, id_{mac}, name_{new}]\!] \\
&\mathsf{expand}[\![stx_{macapp}, \xi, \Sigma]\!] \\
&\quad = \mathsf{expand}[\![\mathsf{mark}[\![stx_{exp}, mrk_{new}]\!], \xi, \Sigma_1]\!] \\
&\quad \text{where } (\textbf{Stx}\ (\textbf{List}\ id_{mac}\ stx_{arg}\ \ldots)\ ctx) = stx_{macapp},\ val = \xi(\mathsf{resolve}[\![id_{mac}, \Sigma]\!]),\ mrk_{new} = \mathsf{fresh}, \\
&\qquad \langle stx_{exp}, \xi_1, \Sigma_1\rangle = \mathsf{eval}[\![(\textbf{App}\ val\ \mathsf{mark}[\![stx_{macapp}, mrk_{new}]\!]), \xi, mrk_{new}, \Sigma]\!] \\
&\mathsf{expand}[\![(\textbf{Stx}\ (\textbf{List}\ id_{stop}\ stx\ \ldots)\ ctx), \xi, \Sigma]\!] \\
&\quad = \langle(\textbf{Stx}\ (\textbf{List}\ id_{stop}\ stx\ \ldots)\ ctx), \Sigma\rangle \\
&\quad \text{where } \textsc{Stop} = \xi(\mathsf{resolve}[\![id_{stop}, \Sigma]\!]) \\
&\mathsf{expand}[\![(\textbf{Stx}\ (\textbf{List}\ stx_{rtor}\ stx_{rnd}\ \ldots)\ ctx), \xi, \Sigma]\!] \\
&\quad = \langle(\textbf{Stx}\ (\textbf{List}\ stx_{exprtor}\ stx_{exprnd}\ \ldots)\ ctx), \Sigma_1\rangle \\
&\quad \text{where } \langle(stx_{exprtor}\ stx_{exprnd}\ \ldots), \Sigma_1\rangle = \mathsf{expand}^*[\![(), (stx_{rtor}\ stx_{rnd}\ \ldots), \xi, \Sigma]\!] \\
&\mathsf{expand}[\![id, \xi, \Sigma]\!] \\
&\quad = \langle id_{new}, \Sigma\rangle \\
&\quad \text{where } (\textsc{Var}\ id_{new}) = \xi(\mathsf{resolve}[\![id, \Sigma]\!])
\end{aligned}$$

$$\begin{aligned}
&\mathsf{expand}^*[\![(stx_{done}\ \ldots), (), \xi, \Sigma]\!] \\
&\quad = \langle(stx_{done}\ \ldots), \Sigma\rangle \\
&\mathsf{expand}^*[\![(stx_{done}\ \ldots), (stx_0\ stx_1\ \ldots), \xi, \Sigma]\!] \\
&\quad = \mathsf{expand}^*[\![(stx_{done}\ \ldots\ stx_{done0}), (stx_1\ \ldots), \xi, \Sigma_1]\!] \\
&\quad \text{where } \langle stx_{done0}, \Sigma_1\rangle = \mathsf{expand}[\![stx_0, \xi, \Sigma]\!]
\end{aligned}$$

$$\begin{aligned}
&\mathsf{parse}[\![(\textbf{Stx}\ (\textbf{List}\ id_{lambda}\ id_{arg}\ stx_{body})\ ctx), \Sigma]\!] \\
&\quad = (\textbf{Fun}\ (\textbf{Var}\ \mathsf{resolve}[\![id_{arg}, \Sigma]\!])\ \mathsf{parse}[\![stx_{body}, \Sigma]\!]) \\
&\quad \text{where } \texttt{lambda} = \mathsf{resolve}[\![id_{lambda}, \Sigma]\!] \\
&\mathsf{parse}[\![(\textbf{Stx}\ (\textbf{List}\ id_{quote}\ stx)\ ctx), \Sigma]\!] \\
&\quad = \mathsf{strip}[\![stx]\!] \\
&\quad \text{where } \texttt{quote} = \mathsf{resolve}[\![id_{quote}, \Sigma]\!] \\
&\mathsf{parse}[\![(\textbf{Stx}\ (\textbf{List}\ id_{syntax}\ stx)\ ctx), \Sigma]\!] \\
&\quad = stx \\
&\quad \text{where } \texttt{syntax} = \mathsf{resolve}[\![id_{syntax}, \Sigma]\!] \\
&\mathsf{parse}[\![(\textbf{Stx}\ (\textbf{List}\ stx_{rator}\ stx_{rand}\ \ldots)\ ctx), \Sigma]\!] \\
&\quad = (\textbf{App}\ \mathsf{parse}[\![stx_{rator}, \Sigma]\!]\ \mathsf{parse}[\![stx_{rand}, \Sigma]\!]\ \ldots) \\
&\mathsf{parse}[\![id, \Sigma]\!] \\
&\quad = (\textbf{Var}\ \mathsf{resolve}[\![id, \Sigma]\!])
\end{aligned}$$

$$\mathsf{resolve}[\![stx, \Sigma]\!] = \mathsf{resolve}^*[\![stx, \Sigma, \varnothing, \varnothing]\!]$$

$$\begin{aligned}
&\mathsf{resolve}^*[\![(\textbf{STX}\ 'name\ \bullet), \Sigma, S_{spine}, S_{branch}]\!] &&= name\\
&\mathsf{resolve}^*[\![(\textbf{STX}\ 'name\ (\textsc{Mark}\ ctx\ mrk)), \Sigma, S_{spine}, S_{branch}]\!] &&= \mathsf{resolve}^*[\![(\textbf{STX}\ 'name\ ctx), \Sigma, S_{spine}, S_{branch}]\!]\\
&\mathsf{resolve}^*[\![(\textbf{STX}\ 'name\ (\textsc{Rename}\ ctx\ id_{orig}\ name_{new}\ \sigma)), \Sigma, S_{spine}, S_{branch}]\!] &&= name_{new}\\
&\quad \text{where } name_1 = \mathsf{resolve}^*[\![id_{orig}, \Sigma, S_{branch}, S_{branch}]\!],\\
&\qquad name_1 = \mathsf{resolve}^*[\![(\textbf{STX}\ 'name\ ctx), \Sigma, \{\sigma\} \cup S_{spine}, S_{branch}]\!],\\
&\qquad \mathsf{marksof}[\![id_{orig}, \Sigma, name_1]\!] = \mathsf{marksof}[\![(\textbf{STX}\ 'name\ ctx), \Sigma, name_1]\!]\\
&\mathsf{resolve}^*[\![(\textbf{STX}\ 'name\ (\textsc{Rename}\ ctx\ id_{orig}\ name_2\ \sigma)), \Sigma, S_{spine}, S_{branch}]\!] &&= \mathsf{resolve}^*[\![(\textbf{STX}\ 'name\ ctx), \Sigma, S_{spine}, S_{branch}]\!]\\
&\mathsf{resolve}^*[\![(\textbf{STX}\ 'name\ (\textsc{Defs}\ ctx\ \sigma)), \Sigma, S_{spine}, S_{branch}]\!] &&= \mathsf{resolve}^*[\![(\textbf{STX}\ 'name\ ctx), \Sigma, S_{spine}, S_{branch}]\!]\\
&\quad \text{where } \sigma \in S_{spine}\\
&\mathsf{resolve}^*[\![(\textbf{STX}\ 'name\ (\textsc{Defs}\ ctx\ \sigma)), \Sigma, S_{spine}, S_{branch}]\!] &&= \mathsf{resolve}^*[\![(\textbf{STX}\ 'name\ ctx_{new}), \Sigma, S_{spine}, \{\sigma\} \cup S_{branch}]\!]\\
&\quad \text{where } \{id \rightarrow name_{new}\ ...\} = \Sigma(\sigma), ctx_{new} = \mathsf{renames}[\![\sigma, ((id\ name_{new})\ ...), ctx]\!]
\end{aligned}$$

$$\begin{aligned}
&\mathsf{renames}[\![\sigma, (), ctx]\!] &&= ctx\\
&\mathsf{renames}[\![\sigma, ((id\ name)\ (id_2\ name_2)\ ...), ctx]\!] &&= \mathsf{renames}[\![\sigma, ((id_2\ name_2)\ ...), (\textsc{Rename}\ ctx\ id\ name\ \sigma)]\!]
\end{aligned}$$

$$\begin{aligned}
&\mathsf{mark}[\![(\textbf{STX}\ atom\ ctx), mrk]\!] &&= (\textbf{STX}\ atom\ (\textsc{Mark}\ ctx\ mrk))\\
&\mathsf{mark}[\![(\textbf{STX}\ (\textbf{LIST}\ stx\ ...)\ ctx), mrk]\!] &&= (\textbf{STX}\ (\textbf{LIST}\ \mathsf{mark}[\![stx, mrk]\!]\ ...)\ (\textsc{Mark}\ ctx\ mrk))\\
&\mathsf{rename}[\![(\textbf{STX}\ atom\ ctx), id, name]\!] &&= (\textbf{STX}\ atom\ (\textsc{Rename}\ ctx\ id\ name\ \textsc{null}))\\
&\mathsf{rename}[\![(\textbf{STX}\ (\textbf{LIST}\ stx\ ...)\ ctx), id, name]\!] &&= (\textbf{STX}\ (\textbf{LIST}\ \mathsf{rename}[\![stx, id, name]\!]\ ...)\ (\textsc{Rename}\ ctx\ id\ name\ \textsc{null}))\\
&\mathsf{defs}[\![(\textbf{STX}\ atom\ ctx), \sigma]\!] &&= (\textbf{STX}\ atom\ (\textsc{Defs}\ ctx\ \sigma))\\
&\mathsf{defs}[\![(\textbf{STX}\ (\textbf{LIST}\ stx\ ...)\ ctx), \sigma]\!] &&= (\textbf{STX}\ (\textbf{LIST}\ \mathsf{defs}[\![stx, \sigma]\!]\ ...)\ (\textsc{Defs}\ ctx\ \sigma))
\end{aligned}$$

$$\begin{aligned}
&\mathsf{marksof}[\![(\textbf{STX}\ val\ \bullet), \Sigma, name]\!] &&= ()\\
&\mathsf{marksof}[\![(\textbf{STX}\ val\ (\textsc{Mark}\ ctx\ mrk)), \Sigma, name]\!] &&= mrk \oplus \mathsf{marksof}[\![(\textbf{STX}\ val\ ctx), \Sigma, name]\!]\\
&\mathsf{marksof}[\![(\textbf{STX}\ val\ (\textsc{Rename}\ ctx\ id\ name_2\ \sigma)), \Sigma, name]\!] &&= \mathsf{marksof}[\![(\textbf{STX}\ val\ ctx), \Sigma, name]\!]\\
&\mathsf{marksof}[\![(\textbf{STX}\ val\ (\textsc{Defs}\ ctx\ \sigma)), \Sigma, name]\!] &&= ()\\
&\quad \text{where } name \in \mathrm{rng}(\Sigma(\sigma))\\
&\mathsf{marksof}[\![(\textbf{STX}\ val\ (\textsc{Defs}\ ctx\ \sigma)), \Sigma, name]\!] &&= \mathsf{marksof}[\![(\textbf{STX}\ val\ ctx), \Sigma, name]\!]\\
&mrk_1 \oplus (mrk_1\ mrk_2\ ...) &&= (mrk_2\ ...)\\
&mrk_1 \oplus (mrk_2\ ...) &&= (mrk_1\ mrk_2\ ...)
\end{aligned}$$

$$\begin{aligned}
&\mathsf{strip}[\![(\textbf{STX}\ atom\ ctx)]\!] &&= atom\\
&\mathsf{strip}[\![(\textbf{STX}\ (\textbf{LIST}\ stx\ ...)\ ctx)]\!] &&= (\textbf{LIST}\ \mathsf{strip}[\![stx]\!]\ ...)
\end{aligned}$$

$$\mathsf{nostops}[\![\xi]\!] = \{var \rightarrow transformer \mid \xi(var) = transformer \text{ and } transformer \neq \textsc{Stop}\}$$